Hibernate ORM & JPA — Complete Course

From Object/Relational Impedance Mismatch to JPQL, with Hands-On Code

Prepared for: Hibernate/JPA Deep-Dive Course

Contents

Lesson 1: Introduction to ORM and Its Need **5**

1.1 What is Object/Relational Mapping (ORM)? . . . 5
1.1.1 Persistence . . . 5
1.1.2 Relational Databases . . . 5
1.2 Object-Relational Impedance Mismatch . . . 6
1.2.1 Granularity . . . 6
1.2.2 Subtypes (Inheritance) . . . 6
1.2.3 Identity . . . 7
1.3 An Introduction to Hibernate . . . 7
1.3.1 Why Hibernate? . . . 7
1.3.2 Hibernate Components and Its Architecture . . . 8
1.3.3 Configuration . . . 9
1.3.4 Understanding Where Hibernate Fits in a Java Application . . . 10
Building a Simple Application (Hands-On) . . . 10
Project Structure . . . 10
`pom.xml` . . . 11
`src/main/resources/hibernate.cfg.xml` . . . 11
`Student.java` (Entity) . . . 12
`HibernateUtil.java` (SessionFactory bootstrap) . . . 13
`App.java` (Demonstration) . . . 13
Running It . . . 14
What Just Happened (Tying Back to the Theory) . . . 15
Key Takeaways from Lesson 1 . . . 15

Lesson 2: The Persistence Life Cycle **16**

2.1 Hibernate Object Life Cycle . . . 16
2.1.1 Transient Object . . . 16
2.1.2 Persistent State . . . 17
2.1.3 Removed State . . . 17
2.1.4 Detached State . . . 17
2.2 Persistence Context . . . 18
2.2.1 CRUD (Create, Refresh, Update, and Delete Entities) . . . 18
2.2.2 Loading Entities . . . 19
2.2.3 Saving Entities . . . 20
2.3 Using Session . . . 20
2.3.1 Transaction . . . 21
2.4 Interceptors . . . 21
2.4.1 Creating Interceptors . . . 21
2.4.2 Interceptor API . . . 22
2.5 Hibernate Filters . . . 22

2.6 Hibernate Fetch Strategies . . . 23
Fetch Timing: Eager vs. Lazy . . . 23
Fetch Style: Select vs. Join vs. Subselect vs. Batch . . . 24
Batch Fetching (mitigating N+1) . . . 24
2.7 Hibernate Batch Processing . . . 25
Step 1 — Enable JDBC statement batching in configuration: . . . 25
Step 2 — Periodically flush and clear the persistence context: . . . 25
Bulk Operations Alternative (HQL bulk update/delete — Lesson 5.2.2) . . . 26
Hands-On: Putting It All Together . . . 26
Key Takeaways from Lesson 2 . . . 27

Lesson 3: Hibernate Association and Collection Mapping 28
3.1 Collection Mapping . . . 28
3.1.1 Association Using SET . . . 28
3.2 Unidirectional Associations . . . 29
3.2.1 One to One . . . 29
3.2.2 One to Many . . . 30
3.2.3 Many to Many . . . 30
3.3 Bidirectional Associations . . . 31
3.4 Select N+1 Problem . . . 32
3.5 Hibernate Cascade Types . . . 33
`orphanRemoval` . . . 34
3.6 Hibernate `inverse` Flag . . . 34
3.7 Component Mapping . . . 35
3.8 Inheritance Realization . . . 35
3.8.1 Table-per-Concrete-Class . . . 36
3.8.2 Table-per-Class-Hierarchy (Single Table) . . . 36
3.8.3 Table-per-Subclass (Joined) . . . 37
Hands-On: Complete Association Example . . . 37
Key Takeaways from Lesson 3 . . . 39

Lesson 4: Transaction Management 40
Why Transaction Management Deserves Its Own Lesson . . . 40
4.1 Programmatic Transaction Management . . . 40
Basic Pattern . . . 40
A Reusable Helper (reducing boilerplate) . . . 40
Demonstrating Rollback . . . 41
JPA-style Programmatic Transactions (`EntityManager`/`EntityTransaction`) . . . 42
4.2 Declarative Transaction Management . . . 42
Why Prefer Declarative Management? . . . 42
Setting It Up with Spring + Hibernate . . . 42
Key `@Transactional` Attributes . . . 44
4.3 Using a DataSource for Transaction Management . . . 45
Configuring Hibernate to Use a Pooled DataSource (HikariCP) . . . 45
DataSource + Declarative Transactions: How They Fit Together . . . 46
JTA (Java EE / Jakarta EE Container-Managed Transactions) . . . 47
Key Takeaways from Lesson 4 . . . 47

Lesson 5: Searches and Queries 48
5.1 Hibernate Query Language (HQL) . . . 48
5.1.2 Using Restrictions with HQL . . . 48
5.1.3 Using Named Parameters . . . 49
5.2 Hibernate Query Language (HQL) Performance . . . 49
5.2.1 Lazy Loading (and Its Interaction with HQL) . . . 49

5.2.2 Bulk Updates . . . 50
5.3 Query by Criteria . . . 51
Dynamic Criteria (the real reason to prefer this API) . . . 51
Joins in Criteria . . . 52
5.4 Hibernate Projections . . . 52
HQL Projections . . . 52
Constructor Expressions (type-safe projections — the recommended approach) . . . 52
Aggregate Projections . . . 53
Criteria API Projections . . . 53
Hands-On: Combined Query Demonstration . . . 53
Key Takeaways from Lesson 5 . . . 54

Lesson 6: Caching **55**
6.1 Introduction to Caching . . . 55
6.2 First Level Cache . . . 55
Demonstrating It . . . 55
Scope and Limits . . . 55
`evict()` and `clear()` . . . 56
6.3 Second Level Cache . . . 56
Common Second-Level Cache Providers . . . 56
Enabling the Second-Level Cache (Ehcache example) . . . 56
Cache Concurrency Strategies . . . 57
Demonstrating the Second-Level Cache . . . 58
Query Cache . . . 58
Cache Region Names & Sizing . . . 58
When (Not) to Use the Second-Level Cache . . . 59
Hands-On: Observing Both Cache Levels Together . . . 59
Key Takeaways from Lesson 6 . . . 60

Lesson 7: Java Persistence API (JPA) **61**
7.1 Introduction to JPA . . . 61
7.2 Main JPA Components . . . 61
7.3 Persistence Unit . . . 62
7.4 Entity Classes . . . 63
7.5 Annotations . . . 64
`@GeneratedValue` Strategies in Detail . . . 65
Optimistic Locking with `@Version` . . . 65
7.6 Entity Manager API . . . 66
Injected `EntityManager` in Spring (declarative style, tying back to Lesson 4.2) . . . 67
7.7 JPA – Life Cycle . . . 67
JPA Entity Lifecycle Callbacks . . . 68
7.7.1 Demonstration on Entity Class . . . 69
Key Takeaways from Lesson 7 . . . 70

Lesson 8: JPQL (Java Persistence Query Language) **71**
8.1 Java Persistence Query Language . . . 71
JPQL vs. HQL — Practical Differences . . . 71
Core JPQL Syntax Recap (via `EntityManager`) . . . 71
`@NamedQuery` — Predefined, Validated-at-Startup Queries . . . 72
`getSingleResult()` vs. `getResultList()` . . . 73
Pagination . . . 73
8.2 Demonstration for JPQL . . . 73
Key Takeaways from Lesson 8 . . . 75
Course Complete . . . 75

Addendum: Optimistic and Pessimistic Locking in Hibernate **77**
Why Locking Matters: The Lost Update Problem . . . 77
Part 1: Optimistic Locking . . . 77
The Mechanism: `@Version` . . . 77
Hands-On: Reproducing and Handling an Optimistic Lock Conflict . . . 78
Handling the Conflict Gracefully (typical web-app pattern) . . . 80
Explicit Optimistic Lock Modes (forcing a version check even on read-only paths) 80
Part 2: Pessimistic Locking . . . 80
The Mechanism: Database-Level Row Locks . . . 80
`LockModeType` Options for Pessimistic Locking . . . 81
Hands-On: Reproducing Pessimistic Lock Blocking . . . 81
Avoiding Indefinite Blocking: Lock Timeouts . . . 83
Deadlock Awareness . . . 83
Optimistic vs. Pessimistic: Side-by-Side Comparison . . . 83
Key Takeaways from This Addendum . . . 84

Addendum: The N+1 Query Problem — Deep Dive on Resolution Strategies **85**
What Is the N+1 Query Problem? . . . 85
Reproducing It . . . 85
Why It Happens . . . 85
Detecting the N+1 Problem . . . 85
Resolution Strategies . . . 86
1. `JOIN FETCH` in HQL/JPQL (the most common, per-query fix) . . . 86
2. JPA Entity Graphs (declarative, reusable, standardized) . . . 86
3. `@BatchSize` (turns N queries into a small constant number) . . . 87
4. `FetchMode.SUBSELECT` . . . 87
5. DTO / Constructor-Expression Projections . . . 88
6. Criteria API Fetch Joins . . . 88
Choosing the Right Strategy . . . 88
Key Takeaways . . . 89

Addendum: Spring Data JPA Repositories and Native SQL Queries **90**
Part 1: Spring Data JPA Repositories . . . 90
Where This Fits in the Request Flow . . . 90
The Core Idea: You Write an Interface, Spring Writes the Implementation . . . 90
Derived Query Methods (query logic generated from the method name) . . . 91
`@Query` — Custom JPQL When Derived Names Get Unwieldy . . . 92
`Pageable` and `Sort` (built-in pagination, wrapping Lesson 8.1's manual approach) 92
What Spring Data JPA Is *Not* . . . 92
Part 2: Native SQL Queries . . . 92
Why Drop Down to Native SQL? . . . 92
Native Queries via `EntityManager` . . . 93
`@SqlResultSetMapping` — Mapping Native SQL to DTOs Precisely . . . 93
Native Queries in Spring Data JPA Repositories . . . 94
Hibernate-Native Escape Hatch (beyond pure JPA) . . . 94
JPQL vs. Native SQL — Decision Guide . . . 94
Key Takeaways from This Addendum . . . 95

Lesson 1: Introduction to ORM and Its Need

1.1 What is Object/Relational Mapping (ORM)?

Object/Relational Mapping (ORM) is a programming technique that lets you map (translate) objects in an object-oriented language (like Java) to rows in a relational database, and back again, automatically.

Instead of writing code like this:

```
String sql = "INSERT INTO STUDENT (ID, NAME, EMAIL) VALUES (?, ?, ?)";
PreparedStatement ps = connection.prepareStatement(sql);
ps.setLong(1, student.getId());
ps.setString(2, student.getName());
ps.setString(3, student.getEmail());
ps.executeUpdate();
```

…an ORM tool lets you write:

```
session.persist(student);
```

The ORM tool (Hibernate, in our case) generates the SQL, manages the JDBC `Connection`, `PreparedStatement`, `ResultSet` mapping, transaction boundaries, and object identity — all behind the scenes.

> Official reference: https://hibernate.org/orm/ — "Hibernate ORM enables you to map plain Java objects to relational database tables, and vice versa, via straightforward configuration."

1.1.1 Persistence

Persistence means data outlives the process that created it. A Java object living in the JVM's heap is **transient** — it disappears when the JVM stops. To make it **persistent**, the object's state must be saved somewhere durable — typically a relational database, but could also be a file, a document store, etc.

In enterprise Java, "persistence" specifically refers to the technology and patterns used to:

1. Save an in-memory object's state to a durable store.
2. Retrieve that state later and reconstruct an equivalent object.
3. Keep the in-memory object and the stored data synchronized (updates, deletes).

Hibernate is a **persistence framework** — its entire job is managing item #1–3 for you.

1.1.2 Relational Databases

Relational databases (Oracle, MySQL, PostgreSQL, H2, SQL Server, etc.) organize data into **tables** made of **rows** and **columns**, related to one another via **foreign keys**, and manipulated via **SQL** (Structured Query Language). Relational theory (Codd's 12 rules) guarantees:

- **Data integrity** via constraints (PK, FK, NOT NULL, UNIQUE, CHECK)
- **Set-based operations** (SQL works on sets of rows, not one object at a time)
- **ACID transactions** (Atomicity, Consistency, Isolation, Durability)
- **Normalization** to reduce redundancy

Java applications are written using **objects** (encapsulation, inheritance, polymorphism, references/graphs). Relational databases store **flat, tabular, set-based data** with no native concept of inheritance or object identity beyond a primary key value. This difference in "world view" is the root of the next topic.

1.2 Object-Relational Impedance Mismatch

The term **"impedance mismatch"** (borrowed from electrical engineering, where mismatched impedance causes signal loss) describes the fundamental conceptual friction between the object-oriented world and the relational world. There are several specific mismatches:

1.2.1 Granularity

An object model can have **fine-grained classes** that don't correspond to a single database table.

Example: A `Person` class might be composed of an `Address` object:

```
public class Person {
    private Long id;
    private String name;
    private Address address;   // a value-typed object, not an entity
}

public class Address {
    private String street;
    private String city;
    private String zipCode;
}
```

In Java, `Address` is its own class (fine granularity, good encapsulation, reusable). In a naive relational design, you might expect a separate `ADDRESS` table — but semantically, `Address` isn't an independent entity; it has no identity of its own and shouldn't be shared across `Person` rows. The relational model doesn't have a built-in notion of "embed this group of columns as a substructure" — you either flatten the columns into the `PERSON` table or create an awkward extra table with a 1:1 relationship.

Hibernate solves this with **Component/Embeddable mapping** (`@Embeddable`, `@Embedded`) — see Lesson 3.7.

1.2.2 Subtypes (Inheritance)

Object-oriented languages support **inheritance** and **polymorphism** naturally:

```
public abstract class Payment { ... }
public class CreditCardPayment extends Payment { ... }
public class BankTransferPayment extends Payment { ... }
```

SQL tables have **no native inheritance concept**. There is no `CREATE TABLE CreditCardPayment EXTENDS Payment`. To store a class hierarchy relationally, you must choose one of several *mapping strategies* — each with different trade-offs in normalization, query simplicity, and polymorphic query support. Hibernate provides three strategies (covered in depth in Lesson 3.8):

- Table-per-concrete-class
- Table-per-class-hierarchy (single table)
- Table-per-subclass (joined subclass)

1.2.3 Identity

This is the subtlest mismatch. In Java, an object has **two notions of sameness**:

- **Object identity**: `a == b` — same memory reference.
- **Object equality**: `a.equals(b)` — logically equivalent per your `equals()` override.

A relational database has exactly **one** notion of sameness: **primary key equality** — two rows are "the same row" if their primary key columns match.

The mismatch: if you load the same database row twice into two separate Java objects (e.g., in two different sessions), you get **two distinct object references** (`a == b` is `false`) even though they represent the same database identity. Conversely, if your `equals()` method is written incorrectly (e.g., compares by name instead of ID), two *different* database rows could appear "equal" in Java even though they are not the same row.

Hibernate addresses this via:

- **Session-scoped identity guarantee**: within a single `Session` (a.k.a. **first-level cache** / **persistence context**), Hibernate guarantees that loading the same row twice returns the **same Java object reference**. This is called the **identity map pattern**.
- It is still the developer's responsibility to implement `equals()`/`hashCode()` correctly (usually based on a natural/business key, not the auto-generated surrogate ID which may be `null` before the object is saved) when entities are compared **across sessions** or placed into Sets/Maps in a detached state.

1.3 An Introduction to Hibernate

1.3.1 Why Hibernate?

Without an ORM, a typical Java data-access layer written in raw JDBC suffers from:

Problem	Description
Boilerplate	Every CRUD operation repeats `Connection` → `PreparedStatement` → `ResultSet` → manual mapping → `close()` in `finally` blocks.
SQL scattered everywhere	SQL strings embedded across the codebase, hard to maintain, hard to port across databases.
Manual object-graph mapping	You must manually map foreign keys to nested objects/collections (joins, lazy loading, N+1 problems handled by hand).
No caching	Every access hits the database.
Database-specific SQL	Moving from MySQL to Oracle often breaks hand-written SQL.
Transaction management by hand	Manual `commit()`/`rollback()`, error-prone.

Hibernate (per hibernate.org) addresses all of this:

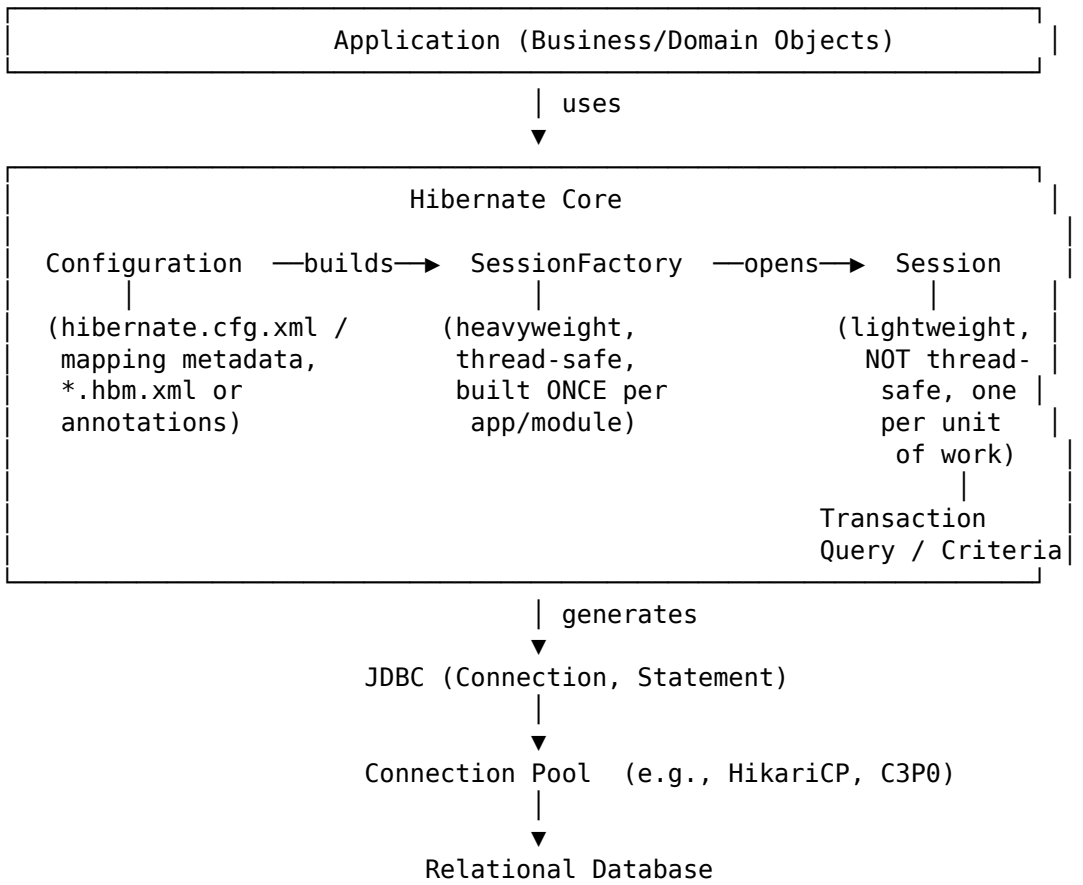
> "Hibernate does the tedious work of mapping your Java objects to database tables so that you can focus on the actual business problem, rather than the infrastructure needed to solve it."

Key benefits:

- **Productivity** – no more hand-written SQL/JDBC boilerplate for standard CRUD.
- **Database portability** – Hibernate generates dialect-specific SQL via a pluggable `Dialect` class.
- **Automatic dirty checking** – Hibernate tracks changes to persistent objects and issues `UPDATE` statements automatically at flush/commit time, without you writing `UPDATE` SQL.
- **Caching** (first-level and second-level) – reduces database round-trips.
- **Transparent lazy loading** – associations are fetched on demand.
- **Standardized query languages** – HQL and Criteria API, both database-agnostic.
- **JPA compliance** – Hibernate is the most widely used implementation of the JPA specification (Lesson 7).

1.3.2 Hibernate Components and Its Architecture

Core building blocks (classic native Hibernate API, pre-JPA naming — you'll see the JPA-equivalent names in Lesson 7):



Key components:

Component	Role
Configuration	Bootstraps Hibernate: reads `hibernate.cfg.xml`, mapping files/annotated classes, builds the metadata model.
SessionFactory	Immutable, thread-safe, expensive-to-create factory for `Session` objects. One per database/application. Holds the second-level cache.
Session	Lightweight, NOT thread-safe, represents a single unit-of-work / conversation with the database. Wraps a JDBC `Connection`. This is the primary interface for CRUD & queries — and it **is** the first-level cache / persistence context.
Transaction	Abstracts the underlying transaction mechanism (JDBC or JTA).
Query / Criteria / HQL	Object-oriented query abstractions that generate SQL.
Dialect	Encapsulates database-specific SQL generation quirks (e.g., `H2Dialect`, `MySQLDialect`, `PostgreSQLDialect`).

1.3.3 Configuration

Hibernate needs to know: (1) how to connect to the database, (2) which dialect of SQL to generate, and (3) which classes/tables to map. Classically this lives in **`hibernate.cfg.xml`**:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-configuration PUBLIC
        "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
        "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
<hibernate-configuration>
    <session-factory>
        <!-- JDBC connection -->
        <property name="hibernate.connection.driver_class">org.h2.Driver</property>
        <property name="hibernate.connection.url">jdbc:h2:mem:testdb;DB_CLOSE_DELAY=-1</propert
        <property name="hibernate.connection.username">sa</property>
        <property name="hibernate.connection.password"></property>

        <!-- SQL dialect -->
        <property name="hibernate.dialect">org.hibernate.dialect.H2Dialect</property>

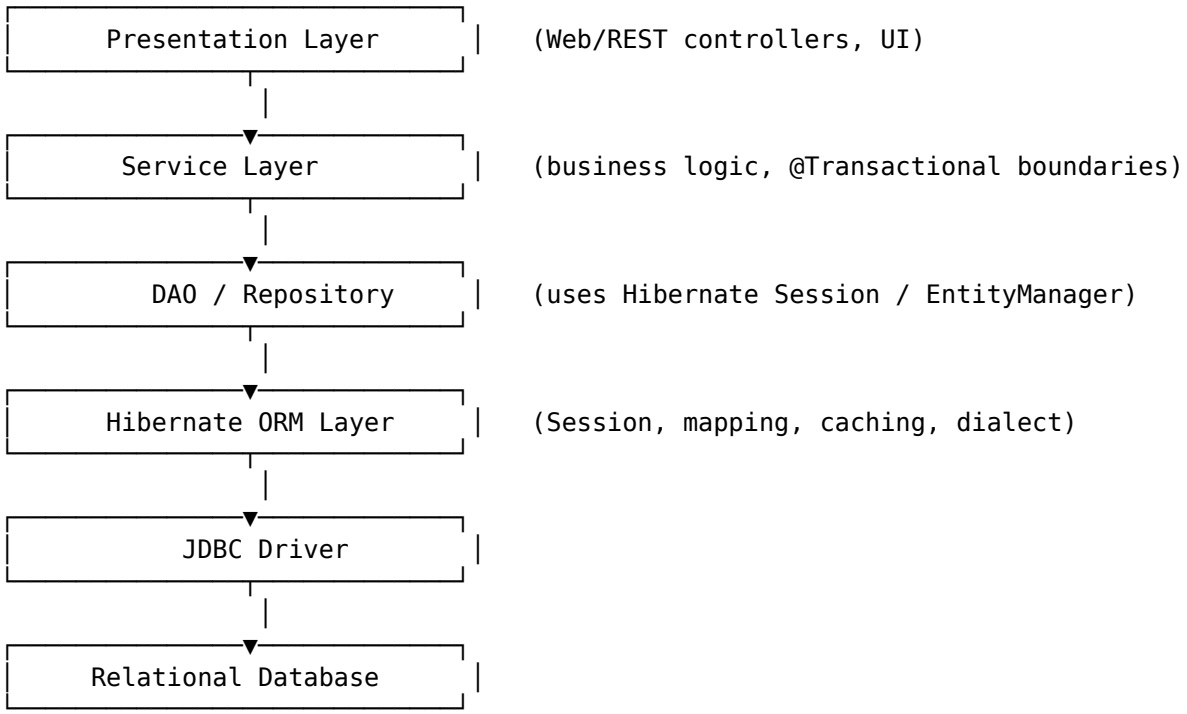
        <!-- Show generated SQL for learning purposes -->
        <property name="hibernate.show_sql">true</property>
        <property name="hibernate.format_sql">true</property>

        <!-- Auto-create/drop schema based on annotated entities (dev only!) -->
        <property name="hibernate.hbm2ddl.auto">update</property>

        <!-- Annotated entity classes -->
        <mapping class="com.example.hibernatecourse.entity.Student"/>
    </session-factory>
</hibernate-configuration>
```

This mirrors the official quick-start shown at https://hibernate.org/orm/documentation/ — modern Hibernate (5.x/6.x) strongly favors **annotations** over XML `hbm.xml` mapping files, while `hibernate.cfg.xml` (or a Java `Configuration`/`ServiceRegistry` bootstrap) is still commonly used to hold connection/dialect settings.

1.3.4 Understanding Where Hibernate Fits in a Java Application



Hibernate sits **between your DAO/Repository layer and JDBC**. Your business/service code never talks to `Connection` or `PreparedStatement` directly — it deals only with plain Java objects (entities) and the `Session`/`EntityManager` API.

Building a Simple Application (Hands-On)

We'll build a minimal, runnable Maven project: a `Student` entity persisted to an in-memory H2 database using pure Hibernate (no Spring, to keep the core concepts visible).

Project Structure

```
hibernate-intro/
├── pom.xml
└── src
    └── main
        ├── java
        │   └── com/example/hibernatecourse
        │       ├── entity/Student.java
        │       ├── util/HibernateUtil.java
        │       └── App.java
        └── resources
            └── hibernate.cfg.xml
```

pom.xml

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
                             http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>com.example</groupId>
    <artifactId>hibernate-intro</artifactId>
    <version>1.0-SNAPSHOT</version>
    <packaging>jar</packaging>

    <properties>
        <maven.compiler.source>17</maven.compiler.source>
        <maven.compiler.target>17</maven.compiler.target>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
        <hibernate.version>6.5.2.Final</hibernate.version>
    </properties>

    <dependencies>
        <!-- Hibernate ORM core -->
        <dependency>
            <groupId>org.hibernate.orm</groupId>
            <artifactId>hibernate-core</artifactId>
            <version>${hibernate.version}</version>
        </dependency>

        <!-- In-memory H2 database, great for learning/tests -->
        <dependency>
            <groupId>com.h2database</groupId>
            <artifactId>h2</artifactId>
            <version>2.2.224</version>
        </dependency>

        <!-- Logging (Hibernate uses SLF4J/JBoss Logging) -->
        <dependency>
            <groupId>org.slf4j</groupId>
            <artifactId>slf4j-simple</artifactId>
            <version>2.0.13</version>
        </dependency>
    </dependencies>
</project>
```

src/main/resources/hibernate.cfg.xml

```xml
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-configuration PUBLIC
        "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
        "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
<hibernate-configuration>
    <session-factory>
```

```
        <property name="hibernate.connection.driver_class">org.h2.Driver</property>
        <property name="hibernate.connection.url">jdbc:h2:mem:testdb;DB_CLOSE_DELAY=-1</propert
        <property name="hibernate.connection.username">sa</property>
        <property name="hibernate.connection.password"></property>

        <property name="hibernate.dialect">org.hibernate.dialect.H2Dialect</property>
        <property name="hibernate.show_sql">true</property>
        <property name="hibernate.format_sql">true</property>
        <property name="hibernate.hbm2ddl.auto">update</property>

        <mapping class="com.example.hibernatecourse.entity.Student"/>
    </session-factory>
</hibernate-configuration>
```

Student.java (Entity)

```
package com.example.hibernatecourse.entity;

import jakarta.persistence.*;

@Entity
@Table(name = "STUDENT")
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "NAME", nullable = false, length = 100)
    private String name;

    @Column(name = "EMAIL", unique = true)
    private String email;

    // Hibernate requires a no-arg constructor
    public Student() {}

    public Student(String name, String email) {
        this.name = name;
        this.email = email;
    }

    // getters and setters
    public Long getId() { return id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }

    @Override
    public String toString() {
        return "Student{id=" + id + ", name='" + name + "', email='" + email + "'}";
    }
```

```
}
```

`HibernateUtil.java` (SessionFactory bootstrap)

```
package com.example.hibernatecourse.util;

import org.hibernate.SessionFactory;
import org.hibernate.cfg.Configuration;

public class HibernateUtil {

    private static final SessionFactory SESSION_FACTORY = buildSessionFactory();

    private static SessionFactory buildSessionFactory() {
        try {
            // Reads hibernate.cfg.xml from classpath and builds the factory.
            return new Configuration()
                    .configure()               // loads hibernate.cfg.xml
                    .buildSessionFactory();
        } catch (Throwable ex) {
            throw new ExceptionInInitializerError(ex);
        }
    }

    public static SessionFactory getSessionFactory() {
        return SESSION_FACTORY;
    }

    public static void shutdown() {
        getSessionFactory().close();
    }
}
```

`App.java` (Demonstration)

```
package com.example.hibernatecourse;

import com.example.hibernatecourse.entity.Student;
import com.example.hibernatecourse.util.HibernateUtil;
import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Transaction;

import java.util.List;

public class App {
    public static void main(String[] args) {
        SessionFactory sessionFactory = HibernateUtil.getSessionFactory();

        // --- CREATE ---
        try (Session session = sessionFactory.openSession()) {
            Transaction tx = session.beginTransaction();
```

```
            Student s1 = new Student("Asha Rao", "asha.rao@example.com");
            Student s2 = new Student("Vikram Singh", "vikram.singh@example.com");

            session.persist(s1);
            session.persist(s2);

            tx.commit();  // INSERT statements are flushed here
            System.out.println("Saved: " + s1 + " and " + s2);
        }

        // --- READ ---
        try (Session session = sessionFactory.openSession()) {
            List<Student> students = session
                    .createQuery("from Student", Student.class)
                    .list();
            students.forEach(System.out::println);
        }

        HibernateUtil.shutdown();
    }
}
```

Running It

```
mvn compile exec:java -Dexec.mainClass="com.example.hibernatecourse.App"
```

Expected console output (abridged):

```
Hibernate:
    insert
    into
        STUDENT
        (EMAIL, NAME)
    values
        (?, ?)
Hibernate:
    insert
    into
        STUDENT
        (EMAIL, NAME)
    values
        (?, ?)
Saved: Student{id=1, name='Asha Rao', email='asha.rao@example.com'} and Student{id=2, ...}
Hibernate:
    select
        s1_0.id,
        s1_0.email,
        s1_0.name
    from
        STUDENT s1_0
Student{id=1, name='Asha Rao', email='asha.rao@example.com'}
Student{id=2, name='Vikram Singh', email='vikram.singh@example.com'}
```

What Just Happened (Tying Back to the Theory)

1. **`Configuration().configure().buildSessionFactory()`** — reads `hibernate.cfg.xml`, resolves the `Student` entity's annotations, and builds a `SessionFactory` — the expensive, thread-safe, one-per-application object from §1.3.2.
2. **`sessionFactory.openSession()`** — opens a lightweight `Session`, wrapping one JDBC `Connection`. This is our unit of work.
3. **`session.persist(s1)`** — makes the transient `Student` object **persistent** (§1.3, and formally covered in Lesson 2). Hibernate doesn't necessarily issue the `INSERT` immediately — it may defer it until flush/commit (see Lesson 2's discussion of the **flush** mechanism).
4. **`tx.commit()`** — triggers a **flush** (SQL is generated & sent to the DB) followed by a JDBC commit, resolving the **granularity** and **identity** mismatches transparently: our fine-grained Java object graph became two flat `INSERT` statements, and Hibernate assigned each `Student` its `IDENTITY`-generated primary key back onto the Java object.
5. **`session.createQuery("from Student", Student.class)`** — this is **HQL** (Hibernate Query Language, Lesson 5), an object-oriented query language that queries against entity names/properties, not table/column names, and Hibernate translates it into the dialect-specific SQL you see in the console (`H2Dialect` here).

This tiny example already demonstrates persistence, ORM mapping, dialect abstraction, and the Configuration → SessionFactory → Session architecture — the foundation for everything in the following lessons.

Key Takeaways from Lesson 1

- ORM bridges the **object-oriented** and **relational** worlds, resolving the **impedance mismatch** (granularity, inheritance, identity).
- Hibernate is the most widely adopted Java ORM, providing productivity, portability, caching, and standardized query languages (HQL, Criteria, and later JPQL).
- Its core architecture: **Configuration → SessionFactory → Session → Transaction/Query**, sitting between your DAO layer and JDBC.
- A working Hibernate app requires: a dialect + connection config (`hibernate.cfg.xml`), an annotated entity class, and a `SessionFactory`/`Session` to perform CRUD.

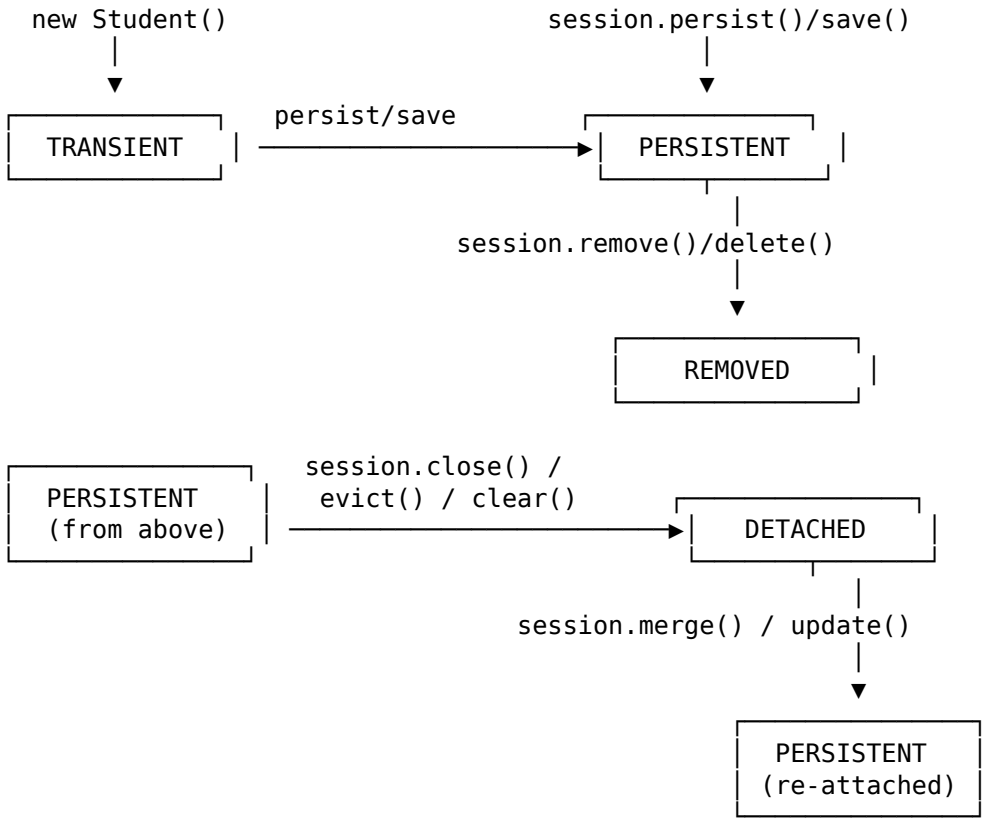
Next: Lesson 2 — The Persistence Life Cycle (transient/persistent/detached/removed states, the persistence context, sessions, interceptors, filters, fetch strategies, and batch processing).

Lesson 2: The Persistence Life Cycle

> This lesson builds on the project from Lesson 1 (`Student` entity, `HibernateUtil`, `hibernate.cfg.xml`). All code below assumes that setup is already in place.

2.1 Hibernate Object Life Cycle

Every entity instance managed by Hibernate exists, at any point in time, in exactly one of **four states**. Understanding these states — and the operations that move an object between them — is the single most important concept in Hibernate, because it explains *when* SQL is actually generated.



2.1.1 Transient Object

A **transient** object is:

- Created with plain Java `new` — `new Student(...)`.
- **Not** associated with any Hibernate `Session`.
- **Not** represented by any row in the database.
- Has no persistent identity (its `@Id` field is typically `null`, unless you set it manually).

```
Student s = new Student("Meera Nair", "meera@example.com"); // TRANSIENT
```

At this point, Hibernate knows nothing about `s`. If the JVM exits now, `s` is simply garbage collected — no database interaction ever occurred.

2.1.2 Persistent State

An object becomes **persistent** when it is associated with an open `Session` (i.e., it is in that Session's **persistence context** — see §2.2). From this point on:

- Hibernate **tracks every change** made to the object's fields (this is called **automatic dirty checking**).
- At flush time, Hibernate compares the object's current state to a snapshot taken when it entered the persistence context, and generates the necessary `UPDATE` SQL for any changed fields — **without you writing any SQL or explicitly calling "update".**
- The object has a valid, non-null persistent identity (primary key value) once inserted.

```
Session session = sessionFactory.openSession();
Transaction tx = session.beginTransaction();

Student s = new Student("Meera Nair", "meera@example.com");
session.persist(s);          // now PERSISTENT — session tracks it

s.setEmail("meera.nair@example.com"); // no explicit SQL call needed!

tx.commit();                 // flush triggers UPDATE (if any dirty fields) + INSERT
session.close();
```

Hibernate will issue only **one** `INSERT` (with the final email value already set before flush) in this specific example, because the change happened before the flush. If you changed `s.setEmail(...)` **after** commit (impossible here since committing closes the transaction) or in a *separate* flush cycle, you'd see a follow-up `UPDATE`. The important conceptual point: **you never call `session.update(s)` to persist changes to an already-persistent object** — Hibernate's dirty-checking does it for you automatically.

2.1.3 Removed State

An object enters the **removed** state when `session.remove(entity)` (JPA-style, Hibernate 6) / `session.delete(entity)` (older Hibernate-native API) is called on a persistent object, within an active transaction:

```
Student s = session.get(Student.class, 1L); // PERSISTENT
session.remove(s);                          // REMOVED — scheduled for DELETE
tx.commit();                                // DELETE SQL issued here
```

The object still exists as a Java reference in memory, but it is **scheduled for deletion**; at flush time, Hibernate issues the `DELETE` statement. After the transaction commits, using `s` further is undefined / discouraged — it no longer corresponds to a database row.

2.1.4 Detached State

An object becomes **detached** when it *was* persistent, but its owning `Session` is closed, cleared, or the object is explicitly evicted — while the object itself still exists as a Java reference with a valid identifier:

```
Student s;
try (Session session = sessionFactory.openSession()) {
    s = session.get(Student.class, 1L); // PERSISTENT
} // session.close() happens here automatically → s becomes DETACHED

s.setName("Updated Name"); // legal Java, but Hibernate is NOT tracking this change anymore!
```

A detached object:

- Still has its identifier.
- Is **no longer tracked** — changes made to it are *not* automatically saved.
- Can be **re-attached** to a new session via `session.merge(s)` (copies its state onto a managed instance, or makes it managed if none exists) or, in older/native APIs, `session.update(s)` (re-attaches the very same instance, riskier if another instance with the same identity already exists in that session, causing a `NonUniqueObjectException`).

```
try (Session session2 = sessionFactory.openSession()) {
    Transaction tx2 = session2.beginTransaction();
    Student managed = session2.merge(s);     // merges detached state into a persistent instanc
    tx2.commit();
}
```

Why detachment matters in real applications: In a typical web app, an entity is often loaded in one transaction (e.g., to render an "edit" form), sent to the browser, and the user's edits come back in a *different* HTTP request — meaning a *different* `Session`. The object is detached the whole time it's "in the browser." `merge()` is exactly the mechanism for reattaching those edits.

2.2 Persistence Context

The **persistence context** is the set of all entity instances currently managed (persistent) within a given `Session`. It's often called the **first-level cache** (covered in depth in Lesson 6.2). Key guarantees it provides:

1. **Identity map**: within one persistence context, loading the same row twice returns the exact same Java object reference (`session.get(Student.class, 1L) == session.get(Student.class, 1L)` is `true`).
2. **Automatic dirty checking**: changes to managed entities are detected and flushed automatically.
3. **Transactional write-behind**: SQL `INSERT`/`UPDATE`/`DELETE` statements are, by default, not sent immediately when you call `persist`/`remove`/mutate a field — they're queued and sent at **flush** time (end of transaction, explicit `session.flush()`, or right before a query that could be affected by pending changes).

A `Session` **is** the persistence context — they are the same thing in Hibernate's native API (in JPA, this is called the `EntityManager`, see Lesson 7).

2.2.1 CRUD (Create, Refresh, Update, and Delete Entities)

Operation	Hibernate-native API	JPA-style API (Hibernate implements this too)	Effect
Create	`session.save(obj)` *(deprecated)*	`session.persist(obj)`	Makes transient object persistent; schedules `INSERT`.
Read	`session.get(Class, id)` / `session.load(Class, id)`	`session.find(Class, id)`	Loads (or returns proxy for) an entity by primary key.

Operation	Hibernate-native API	JPA-style API (Hibernate implements this too)	Effect
Update	(implicit via dirty checking), or `session.update(obj)` for detached	`session.merge(obj)`	Reattaches detached state / propagates changes.
Delete	`session.delete(obj)` *(deprecated)*	`session.remove(obj)`	Schedules `DELETE`.
Refresh	`session.refresh(obj)`	`session.refresh(obj)`	Re-reads the object's state from the database, overwriting in-memory changes — useful when you suspect the DB was changed externally (e.g., by a DB trigger, or another transaction).

```java
try (Session session = sessionFactory.openSession()) {
    Transaction tx = session.beginTransaction();

    // CREATE
    Student s = new Student("Kabir Mehta", "kabir@example.com");
    session.persist(s);
    session.flush(); // force the INSERT now so we can see the generated ID
    Long id = s.getId();

    // READ
    Student loaded = session.get(Student.class, id);
    System.out.println("Loaded: " + loaded);

    // UPDATE (dirty checking — no explicit call needed)
    loaded.setName("Kabir S. Mehta");

    // REFRESH — discard the in-memory name change, re-pull from DB
    session.refresh(loaded);
    System.out.println("After refresh: " + loaded.getName()); // back to "Kabir Mehta"

    // DELETE
    session.remove(loaded);

    tx.commit();
}
```

2.2.2 Loading Entities

Two primary ways to load a single entity by ID, with an important semantic difference:

```java
// 1. get() — hits the database immediately (or 1st/2nd-level cache), returns null if not found
Student s1 = session.get(Student.class, 1L);
if (s1 == null) {
    System.out.println("No student with that ID.");
}

// 2. load() (Hibernate-native only) — returns a PROXY without hitting the DB immediately.
//    Throws ObjectNotFoundException lazily, only when a real field is accessed and no row exis
Student s2 = session.load(Student.class, 1L);
System.out.println(s2.getClass()); // a CGLIB/ByteBuddy-generated proxy subclass, not Student i
System.out.println(s2.getName()); // NOW the SELECT actually fires
```

Use `get()` when you need to check for existence or you'll definitely use the object right away. **Use `load()`** when you only need a reference to set up an association (e.g., `order.setStudent(session.load(Student.class, id))`) and you're confident the row exists — this avoids an unnecessary `SELECT`.

Loading multiple entities uses HQL/Criteria (Lesson 5):

```java
List<Student> all = session.createQuery("from Student", Student.class).list();
```

2.2.3 Saving Entities

```java
Student s = new Student("Divya Iyer", "divya@example.com");

// persist(): the JPA-standard, current recommended method.
// - Guarantees the INSERT happens sometime before transaction commit / next flush.
// - Does NOT guarantee the identifier is populated immediately upon return (depends on generat
session.persist(s);

// save() (Hibernate-native, deprecated as of Hibernate 6):
// - Similar to persist(), but returns the generated identifier immediately, and can be
//   called outside a transaction (not recommended).
// Long id = session.save(s); // legacy style — avoid in new code
```

Best practice (modern Hibernate/JPA style): always prefer `persist()`, `merge()`, `remove()`, `find()` — the JPA-standard method names — over the deprecated Hibernate-native `save()/update()/delete()/get()` pairing, since they behave more predictably and align Hibernate-native code with JPA code (Lesson 7).

2.3 Using Session

The `Session` is your primary handle to the persistence context. Key facts:

- Obtained via `sessionFactory.openSession()`.
- **Not thread-safe** — never share one `Session` across threads.
- Lightweight to create; the expensive object is the `SessionFactory`, created once.
- Wraps exactly one JDBC `Connection` for its lifetime (by default).
- Should be closed after use — always use try-with-resources or a try/finally.

```java
try (Session session = sessionFactory.openSession()) {
    // ... do work ...
} // session.close() called automatically
```

2.3.1 Transaction

Almost all write operations (and often reads, for consistency) should occur **inside an explicit transaction**. Hibernate's `Transaction` interface abstracts over JDBC transactions or JTA (in Java EE/Jakarta EE environments):

```
Session session = sessionFactory.openSession();
Transaction tx = null;
try {
    tx = session.beginTransaction();

    Student s = new Student("Rohan Das", "rohan@example.com");
    session.persist(s);

    tx.commit(); // flush + JDBC commit
} catch (RuntimeException e) {
    if (tx != null) tx.rollback(); // undo everything since beginTransaction()
    throw e;
} finally {
    session.close();
}
```

Why this matters: without an explicit transaction, Hibernate (depending on configuration) may either throw an exception on flush, or silently auto-commit each statement — losing the atomicity guarantee that a batch of related changes either *all* succeed or *all* fail together.

2.4 Interceptors

An **Interceptor** lets you hook into Hibernate's internal lifecycle events (before/after save, load, delete, flush, etc.) to add cross-cutting logic — e.g., auditing (`createdDate`/`updatedDate`), logging, or validation — without polluting your entity or DAO code.

2.4.1 Creating Interceptors

Extend `org.hibernate.EmptyInterceptor` (or implement `org.hibernate.Interceptor` directly) and override only the callbacks you need:

```
package com.example.hibernatecourse.interceptor;

import org.hibernate.EmptyInterceptor;
import org.hibernate.type.Type;
import java.io.Serializable;
import java.time.LocalDateTime;

public class AuditInterceptor extends EmptyInterceptor {

    @Override
    public boolean onSave(Object entity, Serializable id, Object[] state,
                          String[] propertyNames, Type[] types) {
        for (int i = 0; i < propertyNames.length; i++) {
            if ("createdDate".equals(propertyNames[i])) {
                state[i] = LocalDateTime.now();
                return true; // tells Hibernate the state array was modified
            }
```

```
        }
        return false;
    }

    @Override
    public boolean onFlushDirty(Object entity, Serializable id, Object[] currentState,
                                Object[] previousState, String[] propertyNames, Type[] types)
        for (int i = 0; i < propertyNames.length; i++) {
            if ("updatedDate".equals(propertyNames[i])) {
                currentState[i] = LocalDateTime.now();
                return true;
            }
        }
        return false;
    }
}
```

2.4.2 Interceptor API

Common callback methods available on `Interceptor`:

Method	Fires when
`onSave(...)`	Just before an object is saved (INSERT) — lets you mutate state before persisting.
`onFlushDirty(...)`	Just before an UPDATE is issued for a dirty object.
`onDelete(...)`	Just before a DELETE.
`onLoad(...)`	Just after an object is loaded from the DB.
`preFlush(...)` / `postFlush(...)`	Around the entire flush cycle.
`afterTransactionBegin/Completion(...)`	Around transaction boundaries.

Registering an Interceptor (session-scoped, via `SessionBuilder`):

```
Session session = sessionFactory
        .withOptions()
        .interceptor(new AuditInterceptor())
        .openSession();
```

> Note: modern Hibernate/JPA applications generally prefer **JPA Entity Listeners** (`@PrePersist`, `@PreUpdate`, `@PostLoad` — see Lesson 7) or Hibernate **Events** (`org.hibernate.event.spi.*`, registered on the `SessionFactory`'s `EventListenerRegistry`) over the older `Interceptor` API for new code, but `Interceptor` remains fully supported and is common in legacy Hibernate-native codebases.

2.5 Hibernate Filters

Filters let you define reusable, parameterized `WHERE` conditions that Hibernate transparently appends to *every* query against an entity (or collection) — useful for things like soft-deletes, multi-tenancy, or region-based visibility — without editing every single query in your codebase.

Step 1 — Define the filter on the entity:

```
import org.hibernate.annotations.Filter;
import org.hibernate.annotations.FilterDef;
import org.hibernate.annotations.ParamDef;
import jakarta.persistence.*;

@Entity
@Table(name = "STUDENT")
@FilterDef(name = "activeStudentFilter", parameters = @ParamDef(name = "isActive", type = Boole
@Filter(name = "activeStudentFilter", condition = "IS_ACTIVE = :isActive")
public class Student {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
    private String email;

    @Column(name = "IS_ACTIVE")
    private boolean active = true;

    // getters/setters ...
}
```

Step 2 — Enable the filter on a `Session` before querying:

```
try (Session session = sessionFactory.openSession()) {
    session.enableFilter("activeStudentFilter").setParameter("isActive", true);

    List<Student> activeOnly = session.createQuery("from Student", Student.class).list();
    // Hibernate automatically appends: AND IS_ACTIVE = true to the generated SQL
}
```

Filters are **disabled by default** and must be explicitly enabled per-session — they are not a substitute for proper WHERE clauses in ad hoc queries, but shine for cross-cutting concerns like tenant isolation.

2.6 Hibernate Fetch Strategies

Fetching controls *when* and *how* Hibernate retrieves associated entities/collections relative to the owning entity. Two independent dimensions:

Fetch Timing: Eager vs. Lazy

Strategy	Behavior	Default for
EAGER	Associated data is loaded **immediately**, in the same query (via JOIN) or a follow-up SELECT, at the time the owner is loaded.	`@ManyToOne`, `@OneToOne`

Strategy	Behavior	Default for
LAZY	Associated data is loaded **only when accessed** (e.g., calling `student.getCourses().size()`), via a Hibernate-generated proxy/collection wrapper.	`@OneToMany`, `@ManyToMany`

```java
@Entity
public class Student {
    @Id @GeneratedValue
    private Long id;

    @OneToMany(mappedBy = "student", fetch = FetchType.LAZY)
    private Set<Enrollment> enrollments = new HashSet<>();

    @ManyToOne(fetch = FetchType.LAZY)  // override the EAGER default — usually a good idea!
    @JoinColumn(name = "advisor_id")
    private Advisor advisor;
}
```

> **Best practice** (widely recommended on hibernate.org and in the community): default almost everything to `LAZY`, including `@ManyToOne`/`@OneToOne`, and use explicit fetch joins in your queries (HQL `JOIN FETCH`, Lesson 5) when you *know* you'll need the association — this avoids both unnecessary data transfer and the classic **N+1 select problem** (Lesson 3.4).

Fetch Style: Select vs. Join vs. Subselect vs. Batch

Configured via `@Fetch` (Hibernate-specific annotation, `org.hibernate.annotations.Fetch`):

```java
@OneToMany(mappedBy = "student")
@Fetch(FetchMode.SUBSELECT)   // load all students' enrollments via one extra query using a sub
private Set<Enrollment> enrollments;
```

`FetchMode`	SQL Behavior
`SELECT` (default for lazy collections)	One extra `SELECT` **per entity instance** when the collection is accessed — this is the classic N+1 culprit if you loop over many parent entities.
`JOIN`	A single SQL `JOIN` query loads the parent + association together (forces eager loading, ignores `FetchType.LAZY` at the SQL level for that access).
`SUBSELECT`	One extra query using a subselect that fetches the collections for *all* currently loaded parent entities in one shot.

Batch Fetching (mitigating N+1)

```java
@Entity
@BatchSize(size = 20)
public class Student { ... }
```

With `@BatchSize(20)`, if you lazily access the `enrollments` collection on many `Student` instances, Hibernate batches the follow-up `SELECT`s using `WHERE student_id IN (?, ?, ..., up to 20 ids)` instead of one query per student — dramatically reducing round-trips. (Full N+1 discussion in Lesson 3.4.)

2.7 Hibernate Batch Processing

When inserting/updating/deleting **large numbers** of entities, naive use of `persist()` in a loop causes Hibernate to accumulate every entity in the first-level cache (memory grows unbounded) and can also send one network round-trip per row unless JDBC batching is configured.

Step 1 — Enable JDBC statement batching in configuration:

```xml
<property name="hibernate.jdbc.batch_size">50</property>
<property name="hibernate.order_inserts">true</property>
<property name="hibernate.order_updates">true</property>
```

Step 2 — Periodically flush and clear the persistence context:

```java
try (Session session = sessionFactory.openSession()) {
    Transaction tx = session.beginTransaction();

    int batchSize = 50;
    for (int i = 0; i < 1000; i++) {
        Student s = new Student("Student " + i, "student" + i + "@example.com");
        session.persist(s);

        if (i % batchSize == 0 && i > 0) {
            session.flush(); // send the batched INSERTs to the DB
            session.clear(); // detach everything — free up memory (empties 1st-level cache)
        }
    }

    tx.commit();
}
```

Why both `flush()` AND `clear()` are needed:

- `flush()` alone sends the SQL to the database but the persistence context still holds references to all 1000 objects, growing memory usage (`OutOfMemoryError` risk on large datasets).
- `clear()` empties the first-level cache, letting the garbage collector reclaim already-flushed entities.
- Without periodic `flush()`, Hibernate would still eventually flush at commit — but without JDBC-level batching (`hibernate.jdbc.batch_size`), each `INSERT` is still a separate network round trip, just deferred.

Bulk Operations Alternative (HQL bulk update/delete — Lesson 5.2.2)

For pure bulk data changes that don't need entity-level lifecycle callbacks, HQL's bulk `UPDATE/DELETE` bypasses the persistence context entirely and is far more efficient:

```
int updated = session.createMutationQuery(
        "update Student s set s.active = false where s.email like :pattern")
        .setParameter("pattern", "%@old-domain.com")
        .executeUpdate();
```

Hands-On: Putting It All Together

A single runnable demonstration exercising every life-cycle state in one flow:

```
package com.example.hibernatecourse;

import com.example.hibernatecourse.entity.Student;
import com.example.hibernatecourse.util.HibernateUtil;
import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Transaction;

public class LifecycleDemo {
    public static void main(String[] args) {
        SessionFactory sf = HibernateUtil.getSessionFactory();

        // 1. TRANSIENT
        Student s = new Student("Neha Kulkarni", "neha@example.com");
        System.out.println("Transient, id = " + s.getId()); // null

        // 2. TRANSIENT -> PERSISTENT
        Session session1 = sf.openSession();
        Transaction tx1 = session1.beginTransaction();
        session1.persist(s);
        tx1.commit();
        System.out.println("Persistent, id = " + s.getId()); // now populated
        session1.close();

        // 3. PERSISTENT -> DETACHED (session closed above)
        s.setName("Neha K."); // NOT tracked anymore — no SQL will result from this line alone

        // 4. DETACHED -> PERSISTENT (merge)
        Session session2 = sf.openSession();
        Transaction tx2 = session2.beginTransaction();
        Student merged = session2.merge(s); // re-attaches; UPDATE issued at flush since name c
        tx2.commit();
        session2.close();

        // 5. PERSISTENT -> REMOVED
        Session session3 = sf.openSession();
        Transaction tx3 = session3.beginTransaction();
        Student toDelete = session3.get(Student.class, merged.getId());
        session3.remove(toDelete);
```

```
        tx3.commit();
        session3.close();

        System.out.println("Lifecycle demo complete.");
        HibernateUtil.shutdown();
    }
}
```

Running this prints the generated `INSERT`, `UPDATE`, and `DELETE` SQL statements (with `hibernate.show_sql=true` from Lesson 1's config) at exactly the points described above — a great way to visually confirm *when* Hibernate actually talks to the database versus when it's just tracking in-memory state.

Key Takeaways from Lesson 2

- Every entity is **Transient → Persistent → (Detached) → Removed**, and understanding these transitions tells you exactly when SQL fires.
- The **persistence context** (`Session`) is Hibernate's identity map + dirty-checker + write-behind cache — the engine behind automatic `UPDATE` generation.
- Prefer JPA-standard methods (`persist`, `merge`, `remove`, `find`) over deprecated Hibernate-native ones (`save`, `update`, `delete`, `get`/`load` still fine for `get`/`load` specifically).
- **Interceptors** and **Filters** are cross-cutting hooks for auditing and conditional query rewriting.
- **Fetch strategy** (`LAZY` vs `EAGER`, `SELECT`/`JOIN`/`SUBSELECT`, `@BatchSize`) is one of the most important performance levers in Hibernate — get this wrong and you get the N+1 problem (deep-dived in Lesson 3.4).
- **Batch processing** requires periodic `flush()` + `clear()` plus JDBC-level batch-size configuration to scale to large datasets without exhausting memory.

Next: Lesson 3 — Hibernate Association and Collection Mapping (One-to-One, One-to-Many, Many-to-Many, bidirectional mappings, the N+1 problem in depth, cascade types, the `inverse` flag, component mapping, and inheritance mapping strategies).

Lesson 3: Hibernate Association and Collection Mapping

> This lesson extends the `Student` entity from Lessons 1–2 with related entities (`Course`, `Address`, `Advisor`) to demonstrate every association and inheritance mapping style.

3.1 Collection Mapping

Java collections (`Set`, `List`, `Map`, `Bag`/`Collection`) map to a "one" or "many" side of a relationship. Hibernate needs to know **which collection interface** you're using because each has different semantics for duplicates, ordering, and how Hibernate detects changes:

Java Type	Hibernate Behavior	Notes
`Set`	No duplicates, no guaranteed order (unless `@OrderBy`)	Most common choice; maps naturally to relational "no duplicate rows" semantics.
`List`	Ordered, allows duplicates	Requires an `@OrderColumn` (index column) to persist order, or `@OrderBy` to sort on read via SQL `ORDER BY`.
`Map`	Key-value pairs	Requires a `@MapKeyColumn` or `@MapKey`.
`Collection` (a.k.a. "bag")	Unordered, allows duplicates, no index column needed	Simplest to map, but Hibernate can't detect duplicate removal efficiently (may re-insert everything on certain operations).

3.1.1 Association Using SET

`Set` is the most idiomatic default for `@OneToMany`/`@ManyToMany` in Hibernate because it maps cleanly onto SQL semantics (a JOIN naturally returns "no defined order, no artificial duplicates" data), and — critically — `HashSet`'s `equals()`/`hashCode()`-based deduplication interacts safely with Hibernate's dirty-checking, unlike `List`, which needs an explicit order/index column to behave predictably.

```
@Entity
public class Student {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    @OneToMany(mappedBy = "student", cascade = CascadeType.ALL, orphanRemoval = true)
    private Set<Enrollment> enrollments = new HashSet<>();

    // getters/setters, equals/hashCode based on id or a natural key
}
```

> **Important gotcha:** if you override `equals()`/`hashCode()` using mutable fields (like `name`), and that field changes while the object sits inside a `HashSet`, the object can become "lost" in its own bucket. Best practice: base `equals()`/`hashCode()` on a stable

business key, or (simpler, common in practice) rely on `Object`'s default identity-based `equals()`/`hashCode()` and only compare entities by ID explicitly in business logic.

3.2 Unidirectional Associations

Unidirectional means only one side of the relationship has a reference to the other; navigation only works in one direction in Java, even though the database foreign key physically supports lookups both ways via SQL.

3.2.1 One to One

```
@Entity
public class Student {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;

    @OneToOne(cascade = CascadeType.ALL)
    @JoinColumn(name = "address_id") // FK lives in the STUDENT table
    private Address address;
}

@Entity
public class Address {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String street;
    private String city;
    private String zipCode;
    // Address has NO reference back to Student — unidirectional
}
```

Generated schema:

```
CREATE TABLE STUDENT (
    id BIGINT PRIMARY KEY,
    name VARCHAR(255),
    address_id BIGINT REFERENCES ADDRESS(id)
);
CREATE TABLE ADDRESS (
    id BIGINT PRIMARY KEY,
    street VARCHAR(255),
    city VARCHAR(255),
    zip_code VARCHAR(20)
);
```

```
Student s = new Student("Ira Kapoor");
Address a = new Address("221B Baker St", "London", "NW16XE");
s.setAddress(a);

session.persist(s); // cascades and persists Address too, because of CascadeType.ALL
```

3.2.2 One to Many

The "many" side normally owns the foreign key. A **unidirectional One-to-Many** (no `mappedBy`, no back-reference on the child) requires an extra **join table** by default — an important, often-surprising detail:

```
@Entity
public class Student {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;

    @OneToMany(cascade = CascadeType.ALL)
    @JoinColumn(name = "student_id") // avoids the default join table – puts FK directly in COU
    private Set<Course> courses = new HashSet<>();
}

@Entity
public class Course {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String title;
    // no reference back to Student
}
```

Using `@JoinColumn` on a unidirectional `@OneToMany` tells Hibernate to put the foreign key in the child table (`COURSE.student_id`) rather than creating an extra `STUDENT_COURSE` join table — generally the preferred, more efficient mapping.

3.2.3 Many to Many

```
@Entity
public class Student {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;

    @ManyToMany
    @JoinTable(
        name = "STUDENT_COURSE",
        joinColumns = @JoinColumn(name = "student_id"),
        inverseJoinColumns = @JoinColumn(name = "course_id")
    )
    private Set<Course> courses = new HashSet<>();
}

@Entity
public class Course {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String title;
    // unidirectional: no back-reference to Student
}
```

Generated join table:

```sql
CREATE TABLE STUDENT_COURSE (
    student_id BIGINT REFERENCES STUDENT(id),
    course_id BIGINT REFERENCES COURSE(id),
    PRIMARY KEY (student_id, course_id)
);
```

3.3 Bidirectional Associations

In a **bidirectional** association, both sides hold a reference to each other, and Java navigation works both ways (`student.getCourses()` and `course.getStudents()`). The database structure is *identical* to the unidirectional case — bidirectionality is purely an **object-model** convenience. Hibernate requires you to designate exactly one side as the **owning side** (the side that controls the foreign key / join table) via `mappedBy` on the **inverse** side:

```java
@Entity
public class Student {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;

    @ManyToMany(mappedBy = "students") // INVERSE side — Course owns the relationship
    private Set<Course> courses = new HashSet<>();

    // Convenience methods to keep both sides in sync
    public void enroll(Course course) {
        this.courses.add(course);
        course.getStudents().add(this);
    }
}

@Entity
public class Course {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String title;

    @ManyToMany
    @JoinTable(
        name = "STUDENT_COURSE",
        joinColumns = @JoinColumn(name = "course_id"),
        inverseJoinColumns = @JoinColumn(name = "student_id")
    )
    private Set<Student> students = new HashSet<>(); // OWNING side
}
```

Common bidirectional One-to-Many / Many-to-One (the most frequent real-world pattern):

```java
@Entity
public class Student {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
```

```
    @OneToMany(mappedBy = "student", cascade = CascadeType.ALL, orphanRemoval = true)
    private Set<Enrollment> enrollments = new HashSet<>();

    public void addEnrollment(Enrollment e) {
        enrollments.add(e);
        e.setStudent(this); // keep both sides in sync!
    }
}

@Entity
public class Enrollment {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "student_id") // OWNING side — the FK column lives here
    private Student student;

    private String semester;
}
```

> **Golden rule:** the `@ManyToOne` side is (almost) always the owning side, since that's naturally where the foreign key column lives in the table. The `@OneToMany(mappedBy = ...)` side is the inverse/mirror side and should never be written to *without* also updating the owning side — otherwise Hibernate won't persist the relationship correctly (see §3.6, the `inverse` concept).

3.4 Select N+1 Problem

The **N+1 problem** is the single most common Hibernate performance bug. It occurs when you load **N** parent entities, and then, for **each one**, Hibernate lazily fires a *separate* `SELECT` to fetch an association — resulting in **1 + N** total queries instead of a small constant number.

Reproducing it:

```
List<Student> students = session.createQuery("from Student", Student.class).list(); // 1 query

for (Student s : students) {
    System.out.println(s.getName() + " has " + s.getEnrollments().size() + " enrollments");
    // ^ triggers 1 additional SELECT per student, because enrollments is FetchType.LAZY
}
```

Console output shows the smoking gun:

```
select s1_0.id, s1_0.name from STUDENT s1_0                    -- the "1"
select e1_0.student_id, e1_0.id, e1_0.semester
    from ENROLLMENT e1_0 where e1_0.student_id = ?              -- N times, once per student ("N")
select e1_0.student_id, e1_0.id, e1_0.semester
    from ENROLLMENT e1_0 where e1_0.student_id = ?
... (repeated N times)
```

With 1,000 students, that's 1,001 round-trips to the database instead of 1 or 2 — often the #1 cause of slow Hibernate applications in production.

Fixes, from most to least commonly used:

1. **`JOIN FETCH` in HQL** — explicitly eager-load the association for this specific query only:

```
List<Student> students = session.createQuery(
    "select distinct s from Student s left join fetch s.enrollments", Student.class)
    .list(); // now just 1 query total
```

2. **`@BatchSize`** (Lesson 2.6) — turns N queries into `ceil(N / batchSize)` queries using `IN (...)`:

```
@OneToMany(mappedBy = "student")
@BatchSize(size = 25)
private Set<Enrollment> enrollments;
```

3. **`FetchMode.SUBSELECT`** — one extra query total, using a subselect matching all currently loaded parent IDs.
4. **Entity Graphs** (JPA, Lesson 7) — declaratively specify, per query, which associations to eagerly fetch.

Rule of thumb: default associations to `LAZY`, and reach for `JOIN FETCH` / entity graphs on a per-query basis when you know you need the associated data — never flip associations to `EAGER` globally just to avoid N+1, since that just moves the same problem to *every* query that loads that entity, including ones that don't need the association at all.

3.5 Hibernate Cascade Types

Cascading propagates an operation performed on a parent entity to its associated child entities automatically, so you don't have to manually `persist()`/`remove()` every child.

```
@OneToMany(mappedBy = "student", cascade = {CascadeType.PERSIST, CascadeType.MERGE, CascadeType
private Set<Enrollment> enrollments = new HashSet<>();
```

Cascade Type	Effect
`PERSIST`	Saving the parent also saves any transient children.
`MERGE`	Merging the parent also merges detached children.
`REMOVE`	Removing the parent also removes children (issues `DELETE` for each).
`REFRESH`	Refreshing the parent also refreshes children from the DB.
`DETACH`	Detaching the parent also detaches children.
`ALL`	Shorthand for all of the above.

```
Student s = new Student("Farhan Ali");
Enrollment e1 = new Enrollment("Fall 2026");
s.addEnrollment(e1); // sets both sides, see §3.3

session.persist(s); // CascadeType.PERSIST means e1 is persisted too — no need for session.pers
```

`orphanRemoval`

A related-but-distinct setting: `orphanRemoval = true` tells Hibernate to `DELETE` a child automatically when it's **removed from the parent's collection**, even if you never call `remove()` on the child directly:

```
student.getEnrollments().remove(someEnrollment); // with orphanRemoval=true, this alone trigger
```

Without `orphanRemoval`, removing an item from the collection merely unlinks it (sets the FK to `null`, if nullable, or leaves an orphaned row if not) — it does **not** delete the row.

> **Caution:** `CascadeType.REMOVE/ALL` and `orphanRemoval=true` are powerful but dangerous on associations shared across multiple parents (e.g., `@ManyToMany`) — cascading a delete there could unintentionally wipe out data still referenced elsewhere. Reserve aggressive cascading for genuine **parent-owns-child** ("composition") relationships, like `Order` → `OrderLine`, not `Student` ↔ `Course`.

3.6 Hibernate `inverse` Flag

The **`inverse`** concept (historically an explicit XML attribute `inverse="true"` in old `.hbm.xml` mapping files; expressed today via the `mappedBy` attribute in annotations) designates which side of a bidirectional relationship is **responsible for persisting the foreign key**.

- The side **without** `mappedBy` is the **owning side** — Hibernate looks at *this* side's state to decide what SQL (INSERT/UPDATE of the FK column) to generate.
- The side **with** `mappedBy` is the **inverse side** — it is purely for **object-graph navigation in Java**; changes made only on this side are **silently ignored** by Hibernate when generating SQL.

The classic bug this causes:

```
Student s = session.get(Student.class, 1L);
Enrollment e = new Enrollment("Spring 2026");

s.getEnrollments().add(e); // only updates the INVERSE side!
// e.setStudent(s) was never called — the OWNING side (Enrollment.student) is still null

session.persist(e);
tx.commit();
// Result: ENROLLMENT row is inserted with student_id = NULL — NOT what you wanted!
```

Correct approach — always update the **owning side**, ideally via a helper/convenience method (as shown in §3.3's `addEnrollment()`):

```
Enrollment e = new Enrollment("Spring 2026");
e.setStudent(s);       // owning side — sets the actual FK value that will be persisted
s.getEnrollments().add(e); // inverse side — keeps the in-memory Java graph consistent too

session.persist(e);
```

Rule of thumb: *the side with `mappedBy` is inverse/read-only-for-Hibernate; the side without `mappedBy` (usually `@ManyToOne` / the `@JoinColumn` side) is the one whose state actually gets written to the database.*

3.7 Component Mapping

A **component** (JPA calls this an **Embeddable**) is a class with no identity of its own, fully "owned" and embedded within a parent entity's table — directly solving the **granularity mismatch** from Lesson 1.2.1.

```
@Embeddable
public class Address {
    private String street;
    private String city;
    private String zipCode;

    public Address() {}
    public Address(String street, String city, String zipCode) {
        this.street = street; this.city = city; this.zipCode = zipCode;
    }
    // getters/setters
}

@Entity
public class Student {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;

    @Embedded
    private Address address; // NOT a separate table — columns are flattened into STUDENT
}
```

Generated schema — **no separate ADDRESS table**:

```
CREATE TABLE STUDENT (
    id BIGINT PRIMARY KEY,
    name VARCHAR(255),
    street VARCHAR(255),
    city VARCHAR(255),
    zip_code VARCHAR(255)
);
```

Column names can be customized with `@AttributeOverrides` if a `Student` needed *two* addresses (e.g., billing and shipping) using the same `Address` embeddable class, to avoid column name collisions.

3.8 Inheritance Realization

Recall from Lesson 1.2.2: SQL has no native inheritance. Hibernate/JPA offers three `@Inheritance` strategies.

Example hierarchy used below:

```
public abstract class Payment {
    @Id @GeneratedValue private Long id;
    private BigDecimal amount;
    private LocalDate paymentDate;
}
```

```java
public class CreditCardPayment extends Payment {
    private String cardNumber;
    private String cardHolderName;
}
public class BankTransferPayment extends Payment {
    private String iban;
    private String bankName;
}
```

3.8.1 Table-per-Concrete-Class

```java
@Entity
@Inheritance(strategy = InheritanceType.TABLE_PER_CLASS)
public abstract class Payment { ... }
```

Generates **one table per concrete subclass**, each with *all* inherited + own columns (no shared parent table):

```sql
CREATE TABLE CREDIT_CARD_PAYMENT (id BIGINT PRIMARY KEY, amount DECIMAL, payment_date DATE, car
CREATE TABLE BANK_TRANSFER_PAYMENT (id BIGINT PRIMARY KEY, amount DECIMAL, payment_date DATE, i
```

- [+] No joins needed to query a single concrete subtype.
- [-] Polymorphic queries (`from Payment`) require a `UNION ALL` across all tables — can be slow.
- [-] No shared primary key sequence by default (each table needs its own ID generation, or a shared sequence).

3.8.2 Table-per-Class-Hierarchy (Single Table)

```java
@Entity
@Inheritance(strategy = InheritanceType.SINGLE_TABLE)
@DiscriminatorColumn(name = "PAYMENT_TYPE", discriminatorType = DiscriminatorType.STRING)
public abstract class Payment { ... }

@Entity
@DiscriminatorValue("CREDIT_CARD")
public class CreditCardPayment extends Payment { ... }

@Entity
@DiscriminatorValue("BANK_TRANSFER")
public class BankTransferPayment extends Payment { ... }
```

One single table for the entire hierarchy, with a **discriminator column** identifying the concrete type, and **nullable columns** for subtype-specific fields:

```sql
CREATE TABLE PAYMENT (
    id BIGINT PRIMARY KEY,
    payment_type VARCHAR(31),   -- discriminator: 'CREDIT_CARD' or 'BANK_TRANSFER'
    amount DECIMAL,
    payment_date DATE,
    card_number VARCHAR(255),     -- NULL for BankTransferPayment rows
    card_holder_name VARCHAR(255),-- NULL for BankTransferPayment rows
    iban VARCHAR(255),             -- NULL for CreditCardPayment rows
```

```
    bank_name VARCHAR(255)          -- NULL for CreditCardPayment rows
);
```

- [+] **Best polymorphic query performance** — no joins or unions, ever; a single `SELECT * FROM PAYMENT` gets everything.
- [-] Many nullable columns as the hierarchy grows (denormalized); can't enforce `NOT NULL` at the DB level for subtype-specific required fields.
- This is Hibernate/JPA's **default strategy** if `@Inheritance` is omitted entirely.

3.8.3 Table-per-Subclass (Joined)

```
@Entity
@Inheritance(strategy = InheritanceType.JOINED)
public abstract class Payment { ... }

@Entity
public class CreditCardPayment extends Payment { ... }

@Entity
public class BankTransferPayment extends Payment { ... }
```

One table per class (including the abstract parent), linked via **shared primary key / foreign key join**:

```
CREATE TABLE PAYMENT (id BIGINT PRIMARY KEY, amount DECIMAL, payment_date DATE);
CREATE TABLE CREDIT_CARD_PAYMENT (id BIGINT PRIMARY KEY REFERENCES PAYMENT(id), card_number VAR
CREATE TABLE BANK_TRANSFER_PAYMENT (id BIGINT PRIMARY KEY REFERENCES PAYMENT(id), iban VARCHAR(
```

- [+] Fully normalized — no nullable "wasted" columns, proper FK/referential integrity per subtype.
- [-] Polymorphic queries and loading a concrete subtype both require a **SQL JOIN** across parent + child tables — more overhead than `SINGLE_TABLE`, though still far better than `TABLE_PER_CLASS`'s `UNION ALL`.
- Generally considered the **best balance** between normalization and query capability, and is a common real-world default when `SINGLE_TABLE`'s nullable-column sprawl becomes a problem.

Strategy	Query Speed (single subtype)	Query Speed (polymorphic)	Normalization	Nullable Columns
`TABLE_PER_CLASS`	Fast	Slow (`UNION ALL`)	Good	None
`SINGLE_TABLE`	Fastest	Fastest	Poor	Many
`JOINED`	Moderate (1 join)	Moderate (joins)	Best	None

Hands-On: Complete Association Example

```
// --- Entities ---

@Entity
```

```java
public class Student {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;

    @Embedded
    private Address address;

    @OneToMany(mappedBy = "student", cascade = CascadeType.ALL, orphanRemoval = true)
    private Set<Enrollment> enrollments = new HashSet<>();

    public Student() {}
    public Student(String name) { this.name = name; }

    public void addEnrollment(Enrollment e) {
        enrollments.add(e);
        e.setStudent(this);
    }
    // getters/setters omitted for brevity
}

@Entity
public class Enrollment {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String semester;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "student_id")
    private Student student;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "course_id")
    private Course course;

    public Enrollment() {}
    public Enrollment(String semester) { this.semester = semester; }
    public void setStudent(Student student) { this.student = student; }
    // getters/setters omitted for brevity
}

@Entity
public class Course {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String title;
    public Course() {}
    public Course(String title) { this.title = title; }
    // getters/setters omitted
}
```

```java
// --- Demonstration ---
try (Session session = sessionFactory.openSession()) {
```

```
    Transaction tx = session.beginTransaction();

    Student s = new Student("Tanvi Deshpande");
    s.setAddress(new Address("12 MG Road", "Pune", "411001"));

    Course course = new Course("Data Structures");
    session.persist(course);

    Enrollment e = new Enrollment("Fall 2026");
    e.setCourse(course);
    s.addEnrollment(e); // sets owning side (Enrollment.student) AND inverse side (Student.enro

    session.persist(s); // cascades to persist Enrollment (CascadeType.ALL)

    tx.commit();
}

// Fetch with JOIN FETCH to avoid N+1
try (Session session = sessionFactory.openSession()) {
    List<Student> students = session.createQuery(
            "select distinct s from Student s left join fetch s.enrollments e left join fetch e
            Student.class).list();
    students.forEach(st -> System.out.println(st.getName() + " -> " + st.getEnrollments().size(
}
```

Key Takeaways from Lesson 3

- Prefer `Set` for collections; use `List`/`Map` only when order or keyed access is genuinely required.
- **Unidirectional** associations navigate one way in Java; **bidirectional** ones navigate both ways, but the database schema is identical either way — bidirectionality is purely an object-model choice.
- Always update the **owning side** of a bidirectional relationship (the side without `mappedBy`) — the **inverse** side is for object navigation only and is ignored by Hibernate's SQL generation.
- The **N+1 problem** is caused by per-entity lazy loading in a loop; fix with `JOIN FETCH`, `@BatchSize`, or `FetchMode.SUBSELECT` — never by blanket-switching to `EAGER`.
- **Cascade types** propagate operations to children; `orphanRemoval` deletes children removed from a collection.
- **`@Embeddable`/`@Embedded`** (component mapping) solves the granularity mismatch by flattening a no-identity class into the owning entity's table.
- Three inheritance strategies — `TABLE_PER_CLASS`, `SINGLE_TABLE` (default), `JOINED` — trade off normalization vs. query simplicity vs. polymorphic query performance.

Next: Lesson 4 — Transaction Management (programmatic vs. declarative transaction management, and using a `DataSource`/connection pool for production-grade transaction handling).

Lesson 4: Transaction Management

Why Transaction Management Deserves Its Own Lesson

A **transaction** is a unit of work that must be **atomic** — either every change in it succeeds and is committed, or none of it is (rollback), preserving the database's consistency (the "A" and "C" in **ACID**). Hibernate never manages transactions *for* you implicitly — you (or a framework like Spring) must always explicitly demarcate where a transaction begins and ends. There are two broad approaches:

- **Programmatic** — your code explicitly calls `beginTransaction()` / `commit()` / `rollback()`.
- **Declarative** — a framework (typically Spring) wraps your method calls in a transaction automatically, based on configuration or annotations (`@Transactional`), so your business logic code never touches transaction APIs directly.

4.1 Programmatic Transaction Management

This is the approach used in Lessons 1–3: explicit, manual transaction demarcation using Hibernate's native `Transaction` API (or JPA's `EntityTransaction`).

Basic Pattern

```
Session session = sessionFactory.openSession();
Transaction tx = null;
try {
    tx = session.beginTransaction();

    Student s = new Student("Aditi Sharma", "aditi@example.com");
    session.persist(s);

    // ... more operations within the same unit of work ...

    tx.commit(); // flush + physical COMMIT
} catch (RuntimeException e) {
    if (tx != null && tx.isActive()) {
        tx.rollback(); // undoes everything since beginTransaction()
    }
    throw e; // re-throw so calling code knows the operation failed
} finally {
    session.close();
}
```

A Reusable Helper (reducing boilerplate)

Since this try/catch/finally pattern repeats constantly, it's common to wrap it in a utility:

```
package com.example.hibernatecourse.util;

import org.hibernate.Session;
import org.hibernate.SessionFactory;
```

```java
import org.hibernate.Transaction;

import java.util.function.Function;

public class TransactionUtil {

    public static <T> T doInTransaction(SessionFactory sessionFactory, Function<Session, T> wor
        Transaction tx = null;
        try (Session session = sessionFactory.openSession()) {
            tx = session.beginTransaction();
            T result = work.apply(session);
            tx.commit();
            return result;
        } catch (RuntimeException e) {
            if (tx != null && tx.isActive()) {
                tx.rollback();
            }
            throw e;
        }
    }
}
```

Usage:

```java
Student saved = TransactionUtil.doInTransaction(sessionFactory, session -> {
    Student s = new Student("Rahul Verma", "rahul@example.com");
    session.persist(s);
    return s;
});
System.out.println("Saved with id: " + saved.getId());
```

Demonstrating Rollback

```java
try {
    TransactionUtil.doInTransaction(sessionFactory, session -> {
        Student s1 = new Student("Good Record", "good@example.com");
        session.persist(s1);

        Student s2 = new Student("Duplicate Email", "good@example.com"); // violates a UNIQUE c
        session.persist(s2);

        return null; // flush on commit will throw a ConstraintViolationException here
    });
} catch (RuntimeException e) {
    System.out.println("Transaction rolled back — NEITHER student was saved: " + e.getMessage()
}
```

Because both `persist()` calls happened **within the same transaction**, the constraint violation on `s2` causes the **entire transaction** — including the perfectly valid `s1` — to roll back. This is exactly the atomicity guarantee transactions exist to provide.

JPA-style Programmatic Transactions (`EntityManager`/`EntityTransaction`)

If you're using the JPA API surface (Lesson 7) rather than native Hibernate `Session`, the equivalent is:

```
EntityManager em = entityManagerFactory.createEntityManager();
EntityTransaction tx = em.getTransaction();
try {
    tx.begin();
    Student s = new Student("Priya Nambiar", "priya@example.com");
    em.persist(s);
    tx.commit();
} catch (RuntimeException e) {
    if (tx.isActive()) tx.rollback();
    throw e;
} finally {
    em.close();
}
```

4.2 Declarative Transaction Management

Declarative transaction management removes transaction-handling code from your business logic entirely — you simply *declare* (via annotation or XML) that a method should run within a transaction, and a framework (almost always **Spring**, when paired with Hibernate) generates a proxy that wraps your method call with begin/commit/rollback logic using **AOP (Aspect-Oriented Programming)**.

Why Prefer Declarative Management?

- **Separation of concerns**: business logic classes never import `Session`/`Transaction` at all.
- **Consistency**: rollback rules (which exceptions trigger a rollback) are configured once, not repeated in every method.
- **Composability**: nested service calls can propagate/join an existing transaction rather than needing each method to manually check "is there already a transaction open?"

Setting It Up with Spring + Hibernate

Dependencies (Maven, added to Lesson 1's `pom.xml`):

```
<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-orm</artifactId>
    <version>6.1.10</version>
</dependency>
<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-context</artifactId>
    <version>6.1.10</version>
</dependency>
```

Java Config — wiring a `LocalSessionFactoryBean` and a `HibernateTransactionManager`:

```java
package com.example.hibernatecourse.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;
import org.springframework.orm.hibernate5.HibernateTransactionManager;
import org.springframework.orm.hibernate5.LocalSessionFactoryBean;
import org.springframework.transaction.PlatformTransactionManager;
import org.springframework.transaction.annotation.EnableTransactionManagement;

import javax.sql.DataSource;
import java.util.Properties;

@Configuration
@EnableTransactionManagement // enables @Transactional processing
@ComponentScan(basePackages = "com.example.hibernatecourse")
public class HibernateConfig {

    @Bean
    public DataSource dataSource() {
        org.apache.commons.dbcp2.BasicDataSource ds = new org.apache.commons.dbcp2.BasicDataSou
        ds.setDriverClassName("org.h2.Driver");
        ds.setUrl("jdbc:h2:mem:testdb;DB_CLOSE_DELAY=-1");
        ds.setUsername("sa");
        ds.setPassword("");
        return ds;
    }

    @Bean
    public LocalSessionFactoryBean sessionFactory(DataSource dataSource) {
        LocalSessionFactoryBean factoryBean = new LocalSessionFactoryBean();
        factoryBean.setDataSource(dataSource);
        factoryBean.setPackagesToScan("com.example.hibernatecourse.entity");

        Properties props = new Properties();
        props.put("hibernate.dialect", "org.hibernate.dialect.H2Dialect");
        props.put("hibernate.hbm2ddl.auto", "update");
        props.put("hibernate.show_sql", "true");
        factoryBean.setHibernateProperties(props);

        return factoryBean;
    }

    @Bean
    public PlatformTransactionManager transactionManager(org.hibernate.SessionFactory sessionFa
        return new HibernateTransactionManager(sessionFactory);
    }
}
```

Service Layer with `@Transactional`:

```java
package com.example.hibernatecourse.service;

import com.example.hibernatecourse.entity.Student;
```

```java
import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.util.List;

@Service
public class StudentService {

    private final SessionFactory sessionFactory;

    public StudentService(SessionFactory sessionFactory) {
        this.sessionFactory = sessionFactory;
    }

    private Session currentSession() {
        return sessionFactory.getCurrentSession(); // Spring binds this to the current transact
    }

    @Transactional
    public Student registerStudent(String name, String email) {
        Student s = new Student(name, email);
        currentSession().persist(s);
        return s; // no explicit commit/rollback anywhere — Spring's proxy handles it
    }

    @Transactional(readOnly = true)
    public List<Student> findAll() {
        return currentSession().createQuery("from Student", Student.class).list();
    }

    @Transactional(rollbackFor = IllegalArgumentException.class)
    public void deactivate(Long studentId) {
        Student s = currentSession().get(Student.class, studentId);
        if (s == null) {
            throw new IllegalArgumentException("No such student"); // triggers rollback
        }
        s.setActive(false); // dirty checking issues the UPDATE automatically at commit
    }
}
```

Notice: **no `Session.beginTransaction()`, no `tx.commit()`, no `try/catch/rollback` anywhere** in the service. Spring's `@Transactional` proxy:

1. Opens a `Session` (or reuses a suspended/current one, per **propagation** rules) and begins a transaction *before* the method executes.
2. Commits automatically if the method returns normally.
3. Rolls back automatically if the method throws an **unchecked (`RuntimeException`)** exception — by default, checked exceptions do **not** trigger rollback unless you configure `rollbackFor`.

Key `@Transactional` Attributes

Attribute	Purpose
`propagation`	How this transaction relates to an existing one — `REQUIRED` (default: join existing or create new), `REQUIRES_NEW` (suspend existing, start a fresh one), `NESTED`, `MANDATORY`, etc.
`isolation`	SQL isolation level (`READ_COMMITTED`, `REPEATABLE_READ`, `SERIALIZABLE` — DB-dependent).
`readOnly`	Hint to Hibernate/JDBC driver to optimize (e.g., skip dirty-checking flush) for read-only work.
`rollbackFor` / `noRollbackFor`	Explicitly control which exception types trigger (or don't trigger) rollback.
`timeout`	Max seconds before the transaction is forcibly rolled back.

4.3 Using a DataSource for Transaction Management

In Lessons 1–3, Hibernate managed its own JDBC `Connection` directly via properties in `hibernate.cfg.xml` (`hibernate.connection.driver_class`, `.url`, `.username`, `.password`). That approach opens a **brand-new physical connection** for every `Session` — fine for learning, unacceptable for a real application under load.

A **`DataSource`** (typically backed by a **connection pool** like HikariCP, C3P0, or Apache DBCP) manages a pool of pre-opened, reusable physical connections, dramatically reducing the overhead and latency of connection creation, and enabling proper integration with container/framework-managed transactions (JTA in Java EE, or Spring's `DataSourceTransactionManager`/`HibernateTransactionManager`).

Configuring Hibernate to Use a Pooled DataSource (HikariCP)

Maven dependency:

```
<dependency>
    <groupId>com.zaxxer</groupId>
    <artifactId>HikariCP</artifactId>
    <version>5.1.0</version>
</dependency>
```

Programmatic bootstrap (native Hibernate, no Spring):

```
package com.example.hibernatecourse.util;

import com.zaxxer.hikari.HikariConfig;
import com.zaxxer.hikari.HikariDataSource;
import org.hibernate.SessionFactory;
import org.hibernate.boot.MetadataSources;
import org.hibernate.boot.registry.StandardServiceRegistryBuilder;
import org.hibernate.cfg.Environment;
import org.hibernate.service.ServiceRegistry;

import java.util.HashMap;
```

```java
import java.util.Map;

public class HibernateDataSourceUtil {

    public static SessionFactory buildSessionFactory() {
        HikariConfig hikariConfig = new HikariConfig();
        hikariConfig.setDriverClassName("org.h2.Driver");
        hikariConfig.setJdbcUrl("jdbc:h2:mem:testdb;DB_CLOSE_DELAY=-1");
        hikariConfig.setUsername("sa");
        hikariConfig.setPassword("");
        hikariConfig.setMaximumPoolSize(10);       // max concurrent physical connections
        hikariConfig.setMinimumIdle(2);            // kept warm even when idle
        hikariConfig.setConnectionTimeout(30_000); // ms to wait for a free connection before f
        hikariConfig.setIdleTimeout(600_000);

        HikariDataSource dataSource = new HikariDataSource(hikariConfig);

        Map<String, Object> settings = new HashMap<>();
        settings.put(Environment.DATASOURCE, dataSource);   // hand Hibernate the pooled DataSo
        settings.put(Environment.DIALECT, "org.hibernate.dialect.H2Dialect");
        settings.put(Environment.SHOW_SQL, "true");
        settings.put(Environment.HBM2DDL_AUTO, "update");

        ServiceRegistry registry = new StandardServiceRegistryBuilder()
                .applySettings(settings)
                .build();

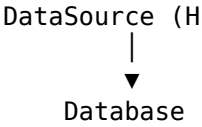
        MetadataSources metadataSources = new MetadataSources(registry)
                .addAnnotatedClass(com.example.hibernatecourse.entity.Student.class);

        return metadataSources.buildMetadata().buildSessionFactory();
    }
}
```

With this in place, every `sessionFactory.openSession()` call **borrows** a physical connection from HikariCP's pool (returning it when the `Session` closes) instead of opening a new TCP connection to the database — the same pattern Spring's `LocalSessionFactoryBean` used in §4.2 (`ds` there was a pooled `BasicDataSource`, easily swapped for `HikariDataSource`).

DataSource + Declarative Transactions: How They Fit Together

```
@Transactional method call
        |
        ▼
Spring's TransactionInterceptor (AOP proxy)
        |  begins transaction via PlatformTransactionManager
        ▼
HibernateTransactionManager
        |  asks SessionFactory for the "current session"
        |  (binds it to the current thread via ThreadLocal)
        ▼
SessionFactory.getCurrentSession()
        |  session borrows a physical Connection from...
        ▼
```



This is precisely why the service-layer code in §4.2 could call `sessionFactory.getCurrentSession()` with no explicit `Session` creation: Spring's `HibernateTransactionManager`, backed by a pooled `DataSource`, opens the `Session`/borrows the `Connection`, binds it to the thread, and makes it available via `getCurrentSession()` for the duration of the `@Transactional` method — then returns the connection to the pool at commit/rollback.

JTA (Java EE / Jakarta EE Container-Managed Transactions)

In full application servers (WildFly, Payara), transactions can instead be managed by the container via **JTA** (Java Transaction API), which supports **distributed/XA transactions** spanning multiple resources (e.g., two different databases, or a database + a JMS queue) — configured via `hibernate.transaction.jta.platform` and a JTA-aware `DataSource` (looked up via JNDI). This is beyond a single-app-server-agnostic tutorial scope, but the conceptual layering is the same: Hibernate delegates the actual "begin/commit/rollback" mechanics to whichever transaction strategy (JDBC-local via `HibernateTransactionManager`, or JTA) is configured.

Key Takeaways from Lesson 4

- **Programmatic** transaction management (`session.beginTransaction()`/`tx.commit()`/`tx.rollback()`) gives full manual control — appropriate for standalone apps, batch jobs, or fine-grained custom logic.
- **Declarative** transaction management (Spring `@Transactional`) removes transaction boilerplate from business logic via AOP proxies — the standard approach in real-world enterprise applications.
- A transaction guarantees **atomicity**: any exception during the transaction rolls back *all* changes made since it began, not just the operation that failed.
- A **`DataSource`** backed by a connection pool (HikariCP, C3P0, DBCP) is essential in production — it avoids the cost of opening a new physical connection per `Session`, and integrates with both programmatic and declarative transaction strategies.
- JTA extends transaction management to distributed/XA scenarios in full Java EE/Jakarta EE containers.

Next: Lesson 5 — Searches and Queries (HQL, restrictions, named parameters, HQL performance, lazy loading, bulk updates, Criteria queries, and Projections).

Lesson 5: Searches and Queries

5.1 Hibernate Query Language (HQL)

HQL is an object-oriented query language: it queries against **entity names and property names**, not table/column names, and is fully database-portable — Hibernate translates HQL into the appropriate SQL dialect at runtime (per the `Dialect` from Lesson 1.3.3).

```
// Note: queries against ENTITY names ("Student"), not table names ("STUDENT")
List<Student> students = session
        .createQuery("from Student", Student.class)
        .list();
```

Compare that to the equivalent raw SQL Hibernate might generate against H2:

```
select s1_0.id, s1_0.email, s1_0.name from STUDENT s1_0
```

HQL supports the full range of query concepts you'd expect: `select`, `where`, `order by`, `group by`, `having`, joins, subqueries, aggregate functions, and even polymorphic queries (querying a superclass returns instances of every subclass, per Lesson 3.8).

```
// Polymorphic query - if Payment has subclasses CreditCardPayment/BankTransferPayment
List<Payment> allPayments = session.createQuery("from Payment", Payment.class).list();
// returns instances of BOTH subclasses transparently
```

5.1.2 Using Restrictions with HQL

"Restrictions" simply means the `WHERE` clause — filtering rows based on conditions:

```
List<Student> topStudents = session.createQuery(
        "from Student s where s.gpa >= 3.5 order by s.gpa desc", Student.class)
        .list();
```

Comparison & logical operators:

```
// Range
session.createQuery("from Student s where s.gpa between 3.0 and 4.0", Student.class).list();

// IN
session.createQuery("from Student s where s.id in (1, 2, 3)", Student.class).list();

// LIKE (pattern matching)
session.createQuery("from Student s where s.name like 'A%'", Student.class).list();

// IS NULL / IS NOT NULL
session.createQuery("from Student s where s.email is not null", Student.class).list();

// Logical combination
session.createQuery(
    "from Student s where s.gpa > 3.0 and (s.active = true or s.email like '%@alumni.edu')",
    Student.class).list();
```

Joining associations in restrictions (implicit vs. explicit join):

```
// Explicit inner join, filtering on the joined entity's property
session.createQuery(
    "select distinct s from Student s join s.enrollments e where e.semester = :sem",
    Student.class)
    .setParameter("sem", "Fall 2026")
    .list();
```

5.1.3 Using Named Parameters

Never concatenate user input directly into HQL strings — this is the HQL equivalent of SQL injection. Always use **named parameters** (`:paramName`) or **positional parameters** (`?1`, `?2`):

```
// GOOD: named parameter — safe, readable, self-documenting
List<Student> results = session.createQuery(
        "from Student s where s.name = :name and s.gpa >= :minGpa", Student.class)
        .setParameter("name", userSuppliedName)
        .setParameter("minGpa", 3.0)
        .list();

// AVOID: string concatenation — vulnerable to HQL injection
// String hql = "from Student s where s.name = '" + userSuppliedName + "'"; // NEVER DO THIS
```

```
// Positional parameters (less common, less readable, but valid)
session.createQuery("from Student s where s.name = ?1 and s.gpa >= ?2", Student.class)
        .setParameter(1, "Aditi Sharma")
        .setParameter(2, 3.0)
        .list();
```

Named parameters also support **collections** directly for `IN` clauses:

```
List<Long> ids = List.of(1L, 2L, 3L);
session.createQuery("from Student s where s.id in :ids", Student.class)
        .setParameter("ids", ids)
        .list();
```

5.2 Hibernate Query Language (HQL) Performance

Writing *correct* HQL is easy; writing *fast* HQL requires understanding what happens after the query executes.

5.2.1 Lazy Loading (and Its Interaction with HQL)

Recall from Lesson 2.6/3.4: `LAZY` associations are proxies that trigger a separate `SELECT` only when accessed. This directly affects how you should write queries:

```
// PROBLEM: N+1, because 'enrollments' is LAZY and accessed in a loop after the query returns
List<Student> students = session.createQuery("from Student", Student.class).list();
for (Student s : students) {
    System.out.println(s.getEnrollments().size()); // fires one SELECT per student!
}
```

```java
// FIX: JOIN FETCH pulls the association eagerly for THIS query only
List<Student> students = session.createQuery(
        "select distinct s from Student s left join fetch s.enrollments", Student.class)
        .list();
for (Student s : students) {
    System.out.println(s.getEnrollments().size()); // no extra SELECT — already loaded
}
```

`distinct` matters here: a SQL `JOIN` against a one-to-many association returns one row *per child*, so without `select distinct`, Hibernate's result list would contain duplicate `Student` references (one per enrollment) at the object level (though Hibernate 6 automatically deduplicates entity roots returned from `JOIN FETCH` at the JPQL/HQL level in many cases — using `distinct` explicitly remains the clear, portable, best-practice way to signal intent).

Multiple `JOIN FETCH`es — be cautious combining more than one **collection** fetch join in a single query (a "MultipleBagFetchException" risk with `List`-typed collections; less of an issue with `Set`, per Lesson 3.1):

```java
// Safe: only ONE collection association is fetch-joined; course is *-to-one so no cartesian bl
session.createQuery(
    "select distinct s from Student s " +
    "left join fetch s.enrollments e " +
    "left join fetch e.course",
    Student.class).list();
```

5.2.2 Bulk Updates

For changing many rows at once, loading each entity into the persistence context (triggering per-row dirty checking and individual `UPDATE` statements) is extremely wasteful. HQL's **bulk update/delete** statements bypass the persistence context and issue a single SQL statement directly:

```java
// Bulk UPDATE — one SQL statement affecting potentially thousands of rows
int updatedCount = session.createMutationQuery(
        "update Student s set s.active = false where s.lastLoginDate < :cutoff")
        .setParameter("cutoff", LocalDate.now().minusYears(1))
        .executeUpdate();

System.out.println("Deactivated " + updatedCount + " students");
```

```java
// Bulk DELETE
int deletedCount = session.createMutationQuery(
        "delete from Enrollment e where e.semester = :oldSemester")
        .setParameter("oldSemester", "Spring 2019")
        .executeUpdate();
```

Critical caveat: bulk HQL updates/deletes operate **directly against the database** and completely bypass the first-level cache (persistence context) — any already-loaded entities in the current `Session` will **not** reflect the change and can become stale/inconsistent with the database until you `refresh()` them or start a fresh `Session`. Bulk operations also **skip entity lifecycle callbacks** (no `@PreUpdate`, no cascading, no `orphanRemoval` logic) since they never actually instantiate the affected rows as Java objects. Use bulk operations for genuine batch/maintenance-style changes, not as a shortcut for regular business-logic updates that need cascading or auditing behavior.

5.3 Query by Criteria

The **Criteria API** builds queries programmatically using a type-safe, object-oriented builder pattern instead of parsing a query string — valuable when query structure varies dynamically based on runtime conditions (e.g., an optional search filter form with many optional fields). Modern Hibernate uses the **JPA Criteria API** (`jakarta.persistence.criteria`), which replaced Hibernate's older, now-deprecated native `Criteria`/`Restrictions` API.

```
import jakarta.persistence.criteria.*;

CriteriaBuilder cb = session.getCriteriaBuilder();
CriteriaQuery<Student> cq = cb.createQuery(Student.class);
Root<Student> root = cq.from(Student.class);

cq.select(root)
  .where(cb.and(
      cb.greaterThanOrEqualTo(root.get("gpa"), 3.0),
      cb.equal(root.get("active"), true)
  ))
  .orderBy(cb.desc(root.get("gpa")));

List<Student> results = session.createQuery(cq).list();
```

Dynamic Criteria (the real reason to prefer this API)

```
public List<Student> search(String namePattern, Double minGpa, Boolean active) {
    CriteriaBuilder cb = session.getCriteriaBuilder();
    CriteriaQuery<Student> cq = cb.createQuery(Student.class);
    Root<Student> root = cq.from(Student.class);

    List<Predicate> predicates = new ArrayList<>();
    if (namePattern != null) {
        predicates.add(cb.like(root.get("name"), namePattern + "%"));
    }
    if (minGpa != null) {
        predicates.add(cb.greaterThanOrEqualTo(root.get("gpa"), minGpa));
    }
    if (active != null) {
        predicates.add(cb.equal(root.get("active"), active));
    }

    cq.select(root).where(predicates.toArray(new Predicate[0]));
    return session.createQuery(cq).list();
}
```

Building the equivalent dynamic query as an HQL string would require fragile string concatenation with conditional `WHERE`/`AND` clauses; the Criteria API expresses the same logic as ordinary, type-checked Java code — errors like a mistyped property name (`"gpaa"`) are only caught with the (older, more verbose) metamodel-based Criteria style, but even the simpler `String`-based `.get("gpa")` style used above still avoids raw string concatenation of *values*, eliminating injection risk entirely.

Joins in Criteria

```
CriteriaQuery<Student> cq = cb.createQuery(Student.class);
Root<Student> root = cq.from(Student.class);
Join<Student, Enrollment> enrollments = root.join("enrollments", JoinType.LEFT);

cq.select(root).distinct(true)
  .where(cb.equal(enrollments.get("semester"), "Fall 2026"));

List<Student> results = session.createQuery(cq).list();
```

5.4 Hibernate Projections

A **projection** returns only *specific columns/expressions* rather than complete entity instances — reducing data transfer and avoiding unnecessary entity hydration when you only need a few fields (e.g., populating a dropdown list, or a summary report).

HQL Projections

```
// Single column projection — returns a List<String>
List<String> names = session.createQuery("select s.name from Student s", String.class).list();

// Multiple columns — returns a List<Object[]> by default
List<Object[]> rows = session.createQuery(
        "select s.name, s.gpa from Student s", Object[].class).list();
for (Object[] row : rows) {
    System.out.println(row[0] + " -> " + row[1]);
}
```

Constructor Expressions (type-safe projections — the recommended approach)

Rather than dealing with untyped `Object[]`, define a simple DTO and use HQL's `new` constructor syntax:

```
public class StudentSummaryDTO {
    private final String name;
    private final double gpa;

    public StudentSummaryDTO(String name, double gpa) { // must match the HQL constructor call
        this.name = name;
        this.gpa = gpa;
    }
    // getters
}
```

```
List<StudentSummaryDTO> summaries = session.createQuery(
        "select new com.example.hibernatecourse.dto.StudentSummaryDTO(s.name, s.gpa) from Stude
        StudentSummaryDTO.class)
        .list();
```

This is the single most useful projection technique in real applications — it's type-safe, avoids loading full entities (and their lazy associations/proxies) into the persistence context, and

integrates cleanly with service/API layers that need DTOs rather than entity objects.

Aggregate Projections

```
// COUNT
Long totalStudents = session.createQuery("select count(s) from Student s", Long.class)
        .uniqueResult();

// AVG / MAX / MIN with GROUP BY
List<Object[]> avgGpaByActiveStatus = session.createQuery(
        "select s.active, avg(s.gpa) from Student s group by s.active", Object[].class)
        .list();
```

Criteria API Projections

```
CriteriaBuilder cb = session.getCriteriaBuilder();
CriteriaQuery<Object[]> cq = cb.createQuery(Object[].class);
Root<Student> root = cq.from(Student.class);

cq.multiselect(root.get("name"), root.get("gpa"))
  .where(cb.isTrue(root.get("active")));

List<Object[]> results = session.createQuery(cq).list();
```

Hands-On: Combined Query Demonstration

```
try (Session session = sessionFactory.openSession()) {
    // 1. HQL with restriction + named parameter
    List<Student> honorRoll = session.createQuery(
            "from Student s where s.gpa >= :minGpa order by s.gpa desc", Student.class)
            .setParameter("minGpa", 3.5)
            .list();
    System.out.println("Honor roll size: " + honorRoll.size());

    // 2. Fetch join to avoid N+1
    List<Student> withEnrollments = session.createQuery(
            "select distinct s from Student s left join fetch s.enrollments", Student.class)
            .list();

    // 3. Bulk update (maintenance-style operation)
    Transaction tx = session.beginTransaction();
    int deactivated = session.createMutationQuery(
            "update Student s set s.active = false where s.gpa < :threshold")
            .setParameter("threshold", 2.0)
            .executeUpdate();
    tx.commit();
    System.out.println("Deactivated: " + deactivated);

    // 4. Criteria-based dynamic search
    CriteriaBuilder cb = session.getCriteriaBuilder();
```

```
    CriteriaQuery<Student> cq = cb.createQuery(Student.class);
    Root<Student> root = cq.from(Student.class);
    cq.select(root).where(cb.like(root.get("name"), "A%"));
    List<Student> namesStartingWithA = session.createQuery(cq).list();

    // 5. DTO projection
    List<StudentSummaryDTO> summaries = session.createQuery(
            "select new com.example.hibernatecourse.dto.StudentSummaryDTO(s.name, s.gpa) from S
            StudentSummaryDTO.class)
            .list();
    summaries.forEach(dto -> System.out.println(dto.getName() + ": " + dto.getGpa()));
}
```

Key Takeaways from Lesson 5

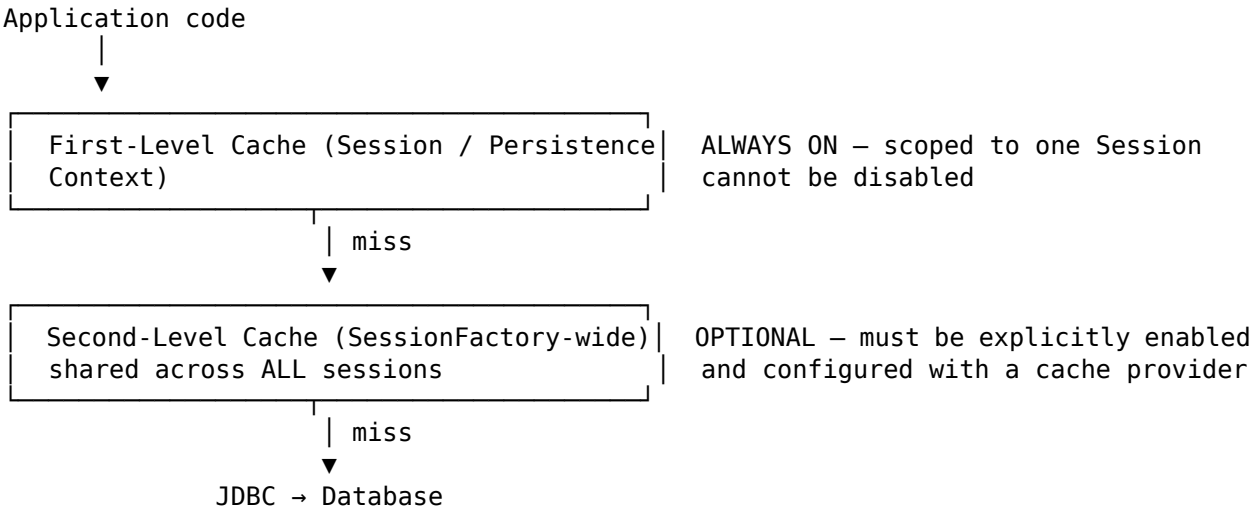
- HQL queries **entities and properties**, not tables/columns — Hibernate translates it into dialect-specific SQL.
- Always use **named (or positional) parameters** — never string-concatenate user input into HQL.
- **`JOIN FETCH`** is the primary HQL-level fix for the N+1 problem from Lesson 3.4, on a per-query basis.
- **Bulk `update/delete`** via `createMutationQuery(...).executeUpdate()` is far more efficient for mass changes, but bypasses the persistence context and entity lifecycle callbacks — use deliberately.
- The **Criteria API** builds type-safe queries programmatically — ideal for queries whose shape/filters vary dynamically at runtime.
- **Projections** (especially DTO **constructor expressions**) let you fetch only what you need, avoiding full entity hydration overhead — a key performance technique for read-heavy/reporting queries.

Next: Lesson 6 — Caching (first-level cache internals recap, and a deep dive into the second-level cache: providers, regions, and concurrency strategies).

Lesson 6: Caching

6.1 Introduction to Caching

Caching stores previously retrieved data in memory so subsequent requests for the same data can be served without a database round-trip. Hibernate operates two distinct cache layers, stacked on top of each other:



Both caches store **entity state** (or query results), keyed by identifier (or query signature), so a repeated lookup by primary key (or an identical query, for the query cache) can be served from memory.

6.2 First Level Cache

The **first-level cache** is simply the **persistence context** you already learned about in Lesson 2.2 — it is **mandatory, always on, and cannot be disabled**. It is scoped to exactly one `Session` and is cleared when that `Session` closes.

Demonstrating It

```
try (Session session = sessionFactory.openSession()) {
    Student s1 = session.get(Student.class, 1L); // SELECT fires — DB hit
    Student s2 = session.get(Student.class, 1L); // NO SELECT — served from 1st-level cache

    System.out.println(s1 == s2); // true — same Java object reference (the identity map, Lesso
}
```

Only **one** `SELECT` appears in the console log despite two `get()` calls — Hibernate recognized the identifier was already loaded into this session's persistence context.

Scope and Limits

```
Student a;
try (Session session1 = sessionFactory.openSession()) {
    a = session1.get(Student.class, 1L); // SELECT fires
}

Student b;
try (Session session2 = sessionFactory.openSession()) {
    b = session2.get(Student.class, 1L); // SELECT fires AGAIN — different session, no shared c
}

System.out.println(a == b); // false — different Session, different first-level cache, differen
```

The first-level cache **never** crosses `Session` boundaries — that's precisely the gap the second-level cache exists to fill.

`evict()` and `clear()`

You can manually remove entries from the first-level cache mid-session (useful in batch processing, Lesson 2.7, to free memory):

```
session.evict(someStudent); // removes just this one entity from the persistence context
session.clear();             // removes EVERYTHING — detaches all managed entities in this sess
```

6.3 Second Level Cache

The **second-level cache (L2 cache)** is **`SessionFactory`-scoped** — shared across *every* `Session` created from that factory, potentially across the entire application (or even across a cluster, depending on the provider). Unlike the first-level cache, it is:

- **Disabled by default** — must be explicitly enabled.
- Backed by a **pluggable external cache provider** (Hibernate itself doesn't implement caching storage; it delegates via the JCache/JSR-107 standard, or a native integration).
- Configured **per-entity** (and optionally per-collection) — you opt individual entities into caching, not the whole domain model at once.

Common Second-Level Cache Providers

Provider	Notes
Ehcache	Most common choice historically; simple, in-JVM or distributed via Terracotta.
Infinispan	JBoss/Red Hat's distributed cache; strong clustering support.
Hazelcast	Popular in-memory data grid, good clustering support.
Caffeine	High-performance local (non-distributed) cache, often used via a JCache wrapper.

Enabling the Second-Level Cache (Ehcache example)

Maven dependencies:

```xml
<dependency>
    <groupId>org.hibernate.orm</groupId>
    <artifactId>hibernate-jcache</artifactId>
    <version>6.5.2.Final</version>
</dependency>
<dependency>
    <groupId>org.ehcache</groupId>
    <artifactId>ehcache</artifactId>
    <version>3.10.8</version>
</dependency>
```

`hibernate.cfg.xml` additions:

```xml
<property name="hibernate.cache.use_second_level_cache">true</property>
<property name="hibernate.cache.use_query_cache">true</property>
<property name="hibernate.cache.region.factory_class">org.hibernate.cache.jcache.internal.JCach
<property name="hibernate.javax.cache.provider">org.ehcache.jsr107.EhcacheCachingProvider</prop
```

Annotate the entity to opt it into the L2 cache:

```java
import jakarta.persistence.*;
import org.hibernate.annotations.Cache;
import org.hibernate.annotations.CacheConcurrencyStrategy;

@Entity
@Cache(usage = CacheConcurrencyStrategy.READ_WRITE)
public class Course {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String title;
    // ...
}
```

Cache Concurrency Strategies

Since the second-level cache is shared across potentially many concurrent sessions/threads (and even multiple application server nodes), you must choose how it behaves under concurrent read/write access:

Strategy	Behavior	Best For
`READ_ONLY`	Data is never updated after being cached; throws an exception if you try to update a cached entity.	Reference/lookup data that never changes (e.g., country codes, category lists).
`NONSTRICT_READ_WRITE`	No locking; a brief window of staleness is possible after updates.	Data updated infrequently, where occasional stale reads are acceptable.
`READ_WRITE`	Uses “soft” locks to prevent stale reads during a concurrent update — the most commonly used strategy for regular mutable entities.	General-purpose mutable entities that need cache consistency guarantees.

Strategy	Behavior	Best For
`TRANSACTIONAL`	Full JTA-integrated transactional cache consistency (requires a JTA-capable cache provider like Infinispan).	Environments requiring the strongest consistency, typically clustered JTA deployments.

Demonstrating the Second-Level Cache

```
// Session 1: loads Course id=1 — SELECT fires, result is now ALSO stored in the L2 cache
try (Session session1 = sessionFactory.openSession()) {
    Course c1 = session1.get(Course.class, 1L);
}

// Session 2 (a DIFFERENT session!): loads Course id=1 again
try (Session session2 = sessionFactory.openSession()) {
    Course c2 = session2.get(Course.class, 1L); // NO SELECT this time — served from the L2 cac
}
```

Unlike the earlier first-level cache example (where `session2` triggered a fresh `SELECT`), enabling the second-level cache on `Course` means the **second session can reuse data loaded by the first** — this is the entire point of L2 caching: amortizing repeated lookups of the same, relatively stable data across the whole application, not just within one unit of work.

Query Cache

By default, even with L2 entity caching enabled, **HQL query *results* are not cached** unless you separately enable the **query cache** (`hibernate.cache.use_query_cache=true`, shown above) and explicitly opt individual queries in:

```
List<Course> popularCourses = session.createQuery(
        "from Course c where c.enrollmentCount > 100", Course.class)
        .setCacheable(true) // opts this specific query's RESULT SET into the query cache
        .list();
```

The query cache stores the **list of matching entity identifiers**, not the full result objects — the actual entity data still comes from the (separately configured) entity-level second-level cache. This means: **the query cache is only useful if the underlying entities are also cached** — otherwise you save the cost of re-running the `WHERE` matching logic in the DB, but still hit the DB once per identifier to hydrate each entity, defeating much of the benefit.

Cache Region Names & Sizing

Each cached entity/collection/query gets its own **cache region**, which you can size and tune independently in the provider's own configuration (e.g., an `ehcache.xml`):

```
<cache alias="com.example.hibernatecourse.entity.Course">
    <expiry>
        <ttl unit="minutes">30</ttl>
    </expiry>
    <resources>
        <heap unit="entries">1000</heap>
    </resources>
</cache>
```

When (Not) to Use the Second-Level Cache

Good candidates: - Reference/lookup data that rarely changes (categories, countries, configuration values). - Data read far more often than it's written, and where slight staleness (for non-`READ_WRITE`/`READ_ONLY` strategies) is tolerable.

Poor candidates: - Frequently updated entities — cache invalidation overhead can outweigh the benefit, and `READ_WRITE`'s soft-locking adds complexity. - Very large entities/collections that would consume excessive cache memory relative to their access frequency. - Data requiring strict, immediate consistency across a cluster where cache synchronization latency is unacceptable.

> As emphasized on hibernate.org's documentation: the second-level cache is a **performance optimization**, not a correctness mechanism — always design your domain model and transactions assuming caching is either absent or occasionally stale (for non-strict strategies), and only enable it deliberately, entity by entity, once you've identified a genuine read-heavy hotspot via profiling.

Hands-On: Observing Both Cache Levels Together

```
public class CachingDemo {
    public static void main(String[] args) {
        SessionFactory sf = HibernateUtil.getSessionFactory(); // L2 cache enabled per config a

        // --- First-level cache demo ---
        try (Session session = sf.openSession()) {
            System.out.println("--- First call in same session ---");
            Course c1 = session.get(Course.class, 1L); // SELECT
            System.out.println("--- Second call in same session (1st-level cache) ---");
            Course c2 = session.get(Course.class, 1L); // no SELECT
            System.out.println("Same reference? " + (c1 == c2)); // true
        }

        // --- Second-level cache demo ---
        System.out.println("--- New session, same entity (2nd-level cache) ---");
        try (Session session2 = sf.openSession()) {
            Course c3 = session2.get(Course.class, 1L); // no SELECT — served from L2 cache
            System.out.println("Loaded from L2 cache: " + c3.getTitle());
        }

        // --- Cache statistics ---
        var stats = sf.getStatistics();
        stats.setStatisticsEnabled(true);
        System.out.println("L2 cache hit count: " + stats.getSecondLevelCacheHitCount());
        System.out.println("L2 cache miss count: " + stats.getSecondLevelCacheMissCount());

        HibernateUtil.shutdown();
    }
}
```

> Enable `hibernate.generate_statistics=true` in your configuration to access `SessionFactory.getStatistics()` — it exposes hit/miss/put counts for both cache levels and is invaluable for verifying that your caching configuration is actually behav-

ing as intended, rather than assuming it based on code alone.

Key Takeaways from Lesson 6

- The **first-level cache** (the persistence context itself) is mandatory, session-scoped, and provides the identity-map guarantee (Lesson 1.2.3 / Lesson 2.2) — it cannot be disabled.
- The **second-level cache** is `SessionFactory`-scoped, shared across all sessions, **disabled by default**, and requires an external provider (Ehcache, Infinispan, Hazelcast, Caffeine) via the JCache integration.
- Enable L2 caching **per entity** with `@Cache`, choosing a concurrency strategy (`READ_ONLY`, `NONSTRICT_READ_WRITE`, `READ_WRITE`, `TRANSACTIONAL`) matched to how often that entity is updated.
- The **query cache** is separate from entity caching and only pays off when the underlying entities are also cached — it stores matching IDs, not full result data.
- Use `SessionFactory.getStatistics()` to verify actual cache hit/miss behavior rather than assuming it.
- Treat the L2 cache purely as a **performance optimization** for read-heavy, rarely-changing data — not a substitute for correct transaction and consistency design.

Next: Lesson 7 — Java Persistence API (JPA) (JPA overview, persistence units, entity classes, annotations, the `EntityManager` API, and the JPA entity lifecycle).

Lesson 7: Java Persistence API (JPA)

7.1 Introduction to JPA

JPA (Jakarta Persistence API, formerly Java Persistence API) is a **specification** — a set of interfaces and annotations defined by Jakarta EE (previously Java EE / Oracle) — describing *how* Java objects should be mapped to relational data and *how* an application should interact with a persistence provider. **JPA itself contains no implementation code** — it's a contract. Hibernate is one of several **JPA providers** that implement this contract (others include EclipseLink and OpenJPA).

This is a crucial distinction that resolves a common point of confusion:

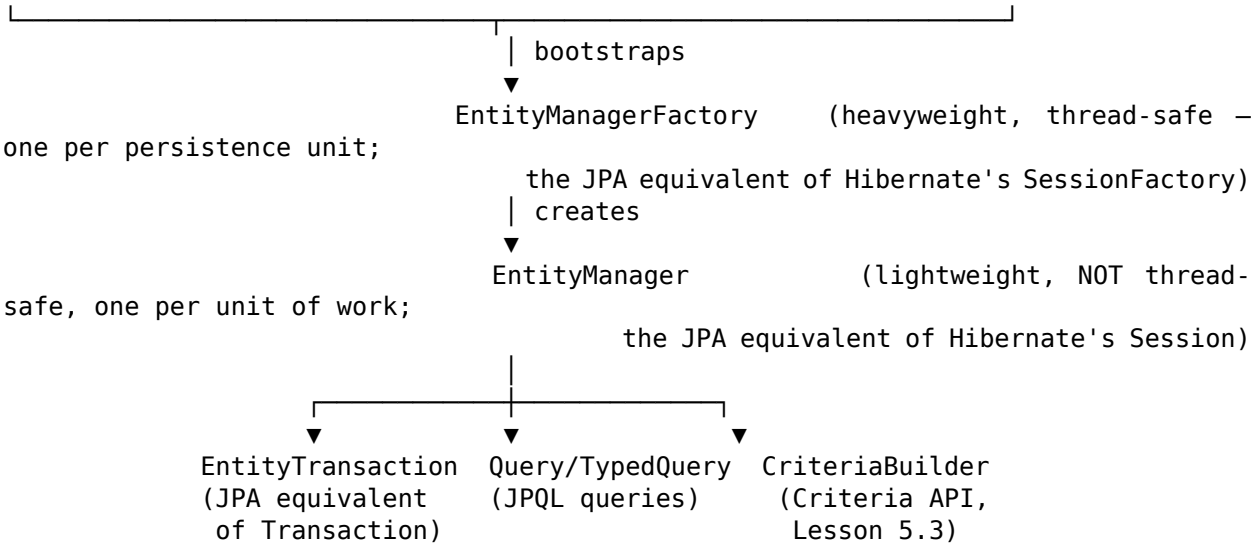
	Hibernate-native API	JPA
What it is	A concrete ORM implementation with its own API surface	A vendor-neutral specification
Core interface	`Session`, `SessionFactory`	`EntityManager`, `EntityManagerFactory`
Query language	HQL (Hibernate Query Language)	JPQL (Java/Jakarta Persistence Query Language) — a subset/standardization of HQL
Portability	Locked to Hibernate	Can (in theory) swap providers (Hibernate ↔ EclipseLink) without rewriting entity/query code
Extra features	Filters, Interceptors, `@BatchSize`, native `Criteria` (deprecated)	Standardized subset; Hibernate still supports its own extensions *alongside* JPA

Everything covered in Lessons 1–6 was demonstrated primarily through Hibernate's **native** API (`Session`, `SessionFactory`, HQL). Hibernate is simultaneously a **full JPA provider** — meaning all of that same functionality is also accessible through the **standardized JPA API** (`EntityManager`), which is what most modern applications (especially Spring Boot / Spring Data JPA projects) actually code against, precisely for the portability and standardization benefits.

> From hibernate.org: "Hibernate ORM is the original implementation of Jakarta Persistence (formerly known as Java Persistence API, or JPA) and, from the beginning, has always aimed to be a state of the art implementation of the specification, but Hibernate also offers an extensive range of features and APIs which go beyond its base implementation of JPA."

7.2 Main JPA Components

```
┌──────────────────────────────────────────────────────────────┐
│  persistence.xml  (defines one or more Persistence Units)    │
```



JPA Component	Hibernate-Native Equivalent	Role
`EntityManagerFactory`	`SessionFactory`	Thread-safe, expensive, one per persistence unit.
`EntityManager`	`Session`	Not thread-safe, cheap, one per unit of work — also **is** the persistence context.
`EntityTransaction`	`Transaction`	Demarcates a unit of work (programmatic style, Lesson 4.1).
`Query` / `TypedQuery<T>`	`Query` (native `Criteria`, deprecated)	Executes JPQL.
`CriteriaBuilder` / `CriteriaQuery`		Type-safe programmatic queries (Lesson 5.3 — this IS the JPA API, already used earlier).

7.3 Persistence Unit

A **persistence unit** is a named configuration bundle — connection details, provider, entity classes, and properties — declared in **`META-INF/persistence.xml`** (placed on the classpath, e.g., `src/main/resources/META-INF/persistence.xml` in a Maven project).

```
<?xml version="1.0" encoding="UTF-8"?>
<persistence xmlns="https://jakarta.ee/xml/ns/persistence"
             xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
             xsi:schemaLocation="https://jakarta.ee/xml/ns/persistence
                 https://jakarta.ee/xml/ns/persistence/persistence_3_0.xsd"
             version="3.0">

    <persistence-unit name="hibernateCoursePU" transaction-type="RESOURCE_LOCAL">
        <provider>org.hibernate.jpa.HibernatePersistenceProvider</provider>

        <class>com.example.hibernatecourse.entity.Student</class>
        <class>com.example.hibernatecourse.entity.Course</class>
```

```
        <class>com.example.hibernatecourse.entity.Enrollment</class>

        <exclude-unlisted-classes>true</exclude-unlisted-classes>

        <properties>
            <property name="jakarta.persistence.jdbc.driver" value="org.h2.Driver"/>
            <property name="jakarta.persistence.jdbc.url" value="jdbc:h2:mem:testdb;DB_CLOSE_DE
            <property name="jakarta.persistence.jdbc.user" value="sa"/>
            <property name="jakarta.persistence.jdbc.password" value=""/>

            <property name="hibernate.dialect" value="org.hibernate.dialect.H2Dialect"/>
            <property name="hibernate.hbm2ddl.auto" value="update"/>
            <property name="hibernate.show_sql" value="true"/>
            <property name="hibernate.format_sql" value="true"/>
        </properties>
    </persistence-unit>
</persistence>
```

`transaction-type` has two possible values:

- `RESOURCE_LOCAL` — the application manages transactions itself (via `EntityTransaction`, §7.6/Lesson 4.1) — used in standalone apps, or Spring apps using `HibernateTransactionManager`/`JpaTransactionManager`.
- `JTA` — a Java EE/Jakarta EE container manages transactions (used inside a full application server, or with Spring's `JtaTransactionManager` for distributed/XA transactions, per Lesson 4.3).

Bootstrapping an `EntityManagerFactory` from this persistence unit:

```
import jakarta.persistence.EntityManagerFactory;
import jakarta.persistence.Persistence;

EntityManagerFactory emf = Persistence.createEntityManagerFactory("hibernateCoursePU");
```

> In a Spring Boot application, you typically never write this bootstrap code yourself — Spring Boot auto-configures the `EntityManagerFactory` from `application.properties`/`application.yml` (`spring.datasource.*`, `spring.jpa.*`) without needing a `persistence.xml` at all. Understanding the underlying `persistence.xml`-driven bootstrap is still valuable for plain Java SE / non-Spring JPA usage, and for understanding what Spring Boot is doing on your behalf.

7.4 Entity Classes

A JPA **entity** is a plain Java class annotated to indicate it should be persisted. The requirements are standardized by the spec (Hibernate's annotation-based native entities already satisfy these, since Hibernate implements JPA):

1. Annotated with `@Entity`.
2. Has a **no-argument constructor** (can be `protected` or `public`).
3. Has a field or property annotated `@Id` (the primary key).
4. Must **not** be declared `final` (JPA providers often generate proxy subclasses for lazy loading — a `final` class cannot be subclassed).
5. Persistent fields/properties should generally not be `final` either, for the same reason.

```java
package com.example.hibernatecourse.entity;

import jakarta.persistence.*;

@Entity
@Table(name = "STUDENT")
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "NAME", nullable = false, length = 100)
    private String name;

    @Column(unique = true)
    private String email;

    public Student() {} // required no-arg constructor

    public Student(String name, String email) {
        this.name = name;
        this.email = email;
    }
    // getters/setters
}
```

This is **exactly** the same `Student` class used since Lesson 1 — because Hibernate's own annotation-based entity mapping *is* the JPA entity mapping standard. There's no separate "JPA version" of the class to write.

7.5 Annotations

The core JPA annotation set (`jakarta.persistence.*`), most of which you've already used throughout this course:

Annotation	Purpose
`@Entity`	Marks a class as a persistent entity.
`@Table(name=...)`	Customizes the table name (defaults to the class name otherwise).
`@Id`	Marks the primary key field.
`@GeneratedValue(strategy=...)`	Configures ID generation: `IDENTITY`, `SEQUENCE`, `TABLE`, `AUTO`.
`@Column(...)`	Customizes column name, nullability, length, uniqueness.
`@Embeddable` / `@Embedded`	Component mapping (Lesson 3.7).
`@OneToOne`, `@OneToMany`, `@ManyToOne`, `@ManyToMany`	Association mappings (Lesson 3).
`@JoinColumn`, `@JoinTable`	Foreign-key/join-table configuration for associations.

Annotation	Purpose
`@Inheritance`, `@DiscriminatorColumn`, `@DiscriminatorValue`	Inheritance mapping (Lesson 3.8).
`@NamedQuery`	Predefines a JPQL query with a name, resolved at startup (fail-fast query validation).
`@Version`	Marks a field used for **optimistic locking** (see below).
`@Transient`	Marks a field that should **not** be persisted at all (pure in-memory/derived state).

`@GeneratedValue` Strategies in Detail

```
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY) // DB auto-increment column (MySQL AUTO_INC
private Long id;

@Id
@GeneratedValue(strategy = GenerationType.SEQUENCE, generator = "student_seq")
@SequenceGenerator(name = "student_seq", sequenceName = "STUDENT_SEQ", allocationSize = 1)
private Long id; // uses a DB SEQUENCE object (common on PostgreSQL/Oracle)

@Id
@GeneratedValue(strategy = GenerationType.AUTO) // lets the provider/dialect pick the best stra
private Long id;
```

`SEQUENCE` generally performs better than `IDENTITY` under Hibernate, because `IDENTITY` forces Hibernate to execute the `INSERT` immediately to learn the generated key (defeating JDBC batching from Lesson 2.7), whereas `SEQUENCE` can pre-fetch a block of IDs and batch inserts freely.

Optimistic Locking with `@Version`

```
@Entity
public class Student {
    @Id @GeneratedValue
    private Long id;

    @Version
    private int version; // Hibernate increments this automatically on every UPDATE

    private String name;
}
```

When two transactions load the same row, and both try to update it, the **second** commit fails with an `OptimisticLockException` because Hibernate includes `WHERE version = <original value>` in the generated `UPDATE`, and the row count returned is `0` (someone else already changed it and bumped the version) — preventing silent lost updates without requiring database-level pessimistic locks for the common case.

7.6 Entity Manager API

The `EntityManager` is JPA's direct equivalent of Hibernate's `Session` — it *is* the persistence context, and provides standardized method names for CRUD (as previewed in Lesson 2.2.1):

```
EntityManagerFactory emf = Persistence.createEntityManagerFactory("hibernateCoursePU");
EntityManager em = emf.createEntityManager();

EntityTransaction tx = em.getTransaction();
try {
    tx.begin();

    // CREATE
    Student s = new Student("Kavya Reddy", "kavya@example.com");
    em.persist(s);

    // READ
    Student found = em.find(Student.class, s.getId());

    // UPDATE — automatic dirty checking, same as Hibernate-native
    found.setName("Kavya R. Reddy");

    // JPQL query
    TypedQuery<Student> query = em.createQuery("select s from Student s where s.name like :patt
    query.setParameter("pattern", "Kavya%");
    List<Student> matches = query.getResultList();

    // DELETE
    Student toRemove = em.find(Student.class, someOtherId);
    if (toRemove != null) {
        em.remove(toRemove);
    }

    tx.commit();
} catch (RuntimeException e) {
    if (tx.isActive()) tx.rollback();
    throw e;
} finally {
    em.close();
}
emf.close();
```

Key `EntityManager` methods:

Method	Purpose
`persist(entity)`	Makes a transient entity persistent (Lesson 2.1.2).
`find(Class, id)`	Loads by primary key, or returns `null` (equivalent to `session.get()`).
`getReference(Class, id)`	Returns a lazy proxy without hitting the DB immediately (equivalent to `session.load()`).
`merge(entity)`	Reattaches a detached entity's state (Lesson 2.1.4).
`remove(entity)`	Schedules deletion (Lesson 2.1.3).

Method	Purpose
`refresh(entity)`	Re-reads state from the DB, discarding in-memory changes.
`flush()`	Forces pending changes to be sent to the DB immediately.
`clear()`	Detaches all managed entities (empties the persistence context).
`createQuery(jpql, Class)`	Creates a `TypedQuery` from JPQL (Lesson 8).
`getCriteriaBuilder()`	Entry point for the JPA Criteria API (Lesson 5.3).
`getTransaction()`	Returns the `EntityTransaction` for programmatic transaction control.
`unwrap(Session.class)`	Escape hatch — retrieves the underlying Hibernate `Session` for native-API features (Filters, Interceptors) not exposed by pure JPA.

Injected `EntityManager` in Spring (declarative style, tying back to Lesson 4.2)

```
@Repository
public class StudentRepository {

    @PersistenceContext
    private EntityManager em; // Spring injects a container-managed, thread-bound EntityManager

    @Transactional
    public Student save(Student s) {
        em.persist(s);
        return s;
    }

    @Transactional(readOnly = true)
    public Optional<Student> findById(Long id) {
        return Optional.ofNullable(em.find(Student.class, id));
    }
}
```

This is precisely how **Spring Data JPA** repositories work under the hood — `JpaRepository`'s built-in `save()`/`findById()`/`delete()` methods are thin wrappers generated around exactly this `EntityManager` usage.

7.7 JPA – Life Cycle

The JPA entity lifecycle uses the **same four states** as native Hibernate (Lesson 2.1) — JPA simply standardizes the terminology and method names:

State	JPA Method to Enter It	Native Hibernate Equivalent
New / Transient	`new Student(...)`	Same (Lesson 2.1.1)
Managed / Persistent	`em.persist(entity)` or loaded via `em.find()`	Same (Lesson 2.1.2)

State	JPA Method to Enter It	Native Hibernate Equivalent
Removed	`em.remove(entity)`	Same (Lesson 2.1.3)
Detached	`em.detach(entity)`, `em.close()`, or `em.clear()`	Same (Lesson 2.1.4)

JPA Entity Lifecycle Callbacks

JPA standardizes lifecycle **callback annotations**, placed directly on entity methods (an alternative to Hibernate's native `Interceptor` API from Lesson 2.4, and the more commonly used approach in modern code):

```java
@Entity
public class Student {
    @Id @GeneratedValue
    private Long id;

    private String name;

    @Column(name = "created_date")
    private LocalDateTime createdDate;

    @Column(name = "updated_date")
    private LocalDateTime updatedDate;

    @PrePersist
    public void onPrePersist() {
        this.createdDate = LocalDateTime.now();
    }

    @PreUpdate
    public void onPreUpdate() {
        this.updatedDate = LocalDateTime.now();
    }

    @PostLoad
    public void onPostLoad() {
        System.out.println("Student " + id + " loaded from DB");
    }

    @PreRemove
    public void onPreRemove() {
        System.out.println("About to delete student " + id);
    }
}
```

Callback	Fires
`@PrePersist`	Before the entity is inserted.
`@PostPersist`	After the entity is inserted.
`@PreUpdate`	Before an update is issued.
`@PostUpdate`	After an update is issued.
`@PreRemove`	Before a delete is issued.
`@PostRemove`	After a delete is issued.

Callback	Fires
`@PostLoad`	After the entity is loaded/refreshed from the database.

External listener classes are also supported via `@EntityListeners`, keeping cross-cutting logic (like auditing) out of the entity class itself — directly analogous to, and generally preferred over, Hibernate's native `Interceptor` (Lesson 2.4) for new JPA-based code:

```
public class AuditListener {
    @PrePersist
    public void prePersist(Object entity) {
        // reflectively set createdDate, or check an Auditable interface, etc.
    }
}

@Entity
@EntityListeners(AuditListener.class)
public class Student { ... }
```

7.7.1 Demonstration on Entity Class

```
public class JpaLifecycleDemo {
    public static void main(String[] args) {
        EntityManagerFactory emf = Persistence.createEntityManagerFactory("hibernateCoursePU");
        EntityManager em = emf.createEntityManager();

        // NEW / TRANSIENT
        Student s = new Student("Ishaan Bose", "ishaan@example.com");
        System.out.println("Transient — id: " + s.getId()); // null

        // NEW -> MANAGED
        em.getTransaction().begin();
        em.persist(s); // @PrePersist fires here, just before INSERT
        em.getTransaction().commit();
        System.out.println("Managed — id: " + s.getId()); // populated

        // MANAGED -> DETACHED
        em.detach(s);
        s.setName("Ishaan B."); // NOT tracked — no SQL results from this alone

        // DETACHED -> MANAGED (merge)
        em.getTransaction().begin();
        Student merged = em.merge(s); // @PreUpdate fires at flush, since name differs
        em.getTransaction().commit();

        // MANAGED -> REMOVED
        em.getTransaction().begin();
        em.remove(merged); // @PreRemove fires just before DELETE
        em.getTransaction().commit();

        em.close();
        emf.close();
```

```
    }
}
```

Key Takeaways from Lesson 7

- **JPA is a specification, not an implementation** — Hibernate is one (the original, most widely used) provider that implements it; you've been using JPA-compatible mappings since Lesson 1 without necessarily realizing it.
- **`EntityManagerFactory`/`EntityManager`** are the standardized JPA equivalents of Hibernate's `SessionFactory`/`Session`, configured via a **persistence unit** (`persistence.xml`), or auto-configured by Spring Boot.
- JPA **entities** have the same structural requirements Hibernate already imposed: `@Entity`, no-arg constructor, `@Id`, non-final class.
- Standard annotations (`@Column`, associations, `@GeneratedValue`, `@Version` for optimistic locking) cover the vast majority of mapping needs portably across providers.
- The **entity lifecycle** (new/managed/removed/detached) is conceptually identical to Hibernate's native transient/persistent/removed/detached states — JPA adds standardized **lifecycle callback annotations** (`@PrePersist`, `@PostLoad`, etc.) as the modern alternative to Hibernate's native `Interceptor` API.

Next: Lesson 8 — JPQL (Java Persistence Query Language) — the standardized query language counterpart to HQL, and a focused demonstration comparing the two.

Lesson 8: JPQL (Java Persistence Query Language)

8.1 Java Persistence Query Language

JPQL is the JPA specification's standardized query language — conceptually and syntactically the near twin of Hibernate's own HQL (Lesson 5.1). In fact, **HQL is a superset of JPQL**: every valid JPQL query is valid HQL, but Hibernate's HQL adds extra syntax and functions (some of the constructs used in Lesson 5, like `createMutationQuery`'s bulk operations or certain Hibernate-specific functions, go slightly beyond the strict JPQL spec, though the overlap is very large in everyday use).

Like HQL, JPQL queries **entity names and property names**, not table/column names, and is executed through the standardized `EntityManager` (Lesson 7.6) rather than Hibernate's native `Session`:

```
TypedQuery<Student> query = em.createQuery(
        "select s from Student s where s.gpa >= :minGpa order by s.gpa desc", Student.class);
query.setParameter("minGpa", 3.5);
List<Student> honorRoll = query.getResultList();
```

JPQL vs. HQL — Practical Differences

Aspect	JPQL	HQL
API entry point	`EntityManager.createQuery(...)`	`Session.createQuery(...)`
Bulk updates/deletes	`executeUpdate()` via `Query`/standard JPQL `UPDATE`/`DELETE`	Same syntax; Hibernate additionally offers `createMutationQuery` as a clearer-named alias (Lesson 5.2.2)
Vendor-specific functions	Not guaranteed portable	Hibernate supports extra built-in functions, native SQL fragments via `function()`, etc.
`@NamedQuery`	Standardized, portable across providers	Also supported, Hibernate-flavored extensions allowed
Portability	Designed to work unchanged if you swap JPA providers	Tied to Hibernate

In everyday development, you will find that **almost all the HQL you wrote in Lesson 5 is already valid JPQL** — restrictions, named parameters, joins, aggregate functions, and constructor expressions (DTO projections) all work identically. The main mental shift is which **object** you call `createQuery` on (`EntityManager` vs. `Session`) and which exception types/transaction API surround it (`EntityTransaction` vs. `Transaction`, per Lesson 7.2).

Core JPQL Syntax Recap (via `EntityManager`)

```
// Restrictions + named parameters (Lesson 5.1.2 / 5.1.3, identical syntax)
List<Student> results = em.createQuery(
        "select s from Student s where s.name like :pattern and s.active = true", Student.class
```

```
        .setParameter("pattern", "A%")
        .getResultList();

// JOIN
List<Student> enrolledInFall = em.createQuery(
        "select distinct s from Student s join s.enrollments e where e.semester = :sem", Studen
        .setParameter("sem", "Fall 2026")
        .getResultList();

// JOIN FETCH (avoiding N+1, Lesson 3.4/5.2.1 — identical concept and syntax)
List<Student> withEnrollments = em.createQuery(
        "select distinct s from Student s left join fetch s.enrollments", Student.class)
        .getResultList();

// Aggregate + GROUP BY
List<Object[]> avgGpaByStatus = em.createQuery(
        "select s.active, avg(s.gpa) from Student s group by s.active", Object[].class)
        .getResultList();

// Constructor expression / DTO projection (Lesson 5.4, identical syntax)
List<StudentSummaryDTO> summaries = em.createQuery(
        "select new com.example.hibernatecourse.dto.StudentSummaryDTO(s.name, s.gpa) from Stude
        StudentSummaryDTO.class)
        .getResultList();

// Bulk UPDATE
em.getTransaction().begin();
int updated = em.createQuery("update Student s set s.active = false where s.gpa < :threshold")
        .setParameter("threshold", 2.0)
        .executeUpdate();
em.getTransaction().commit();
```

`@NamedQuery` — Predefined, Validated-at-Startup Queries

A JPA-standard way to declare a query **on the entity itself**, giving it a name for later lookup — the provider validates the JPQL syntax at `EntityManagerFactory`/`SessionFactory` bootstrap time (fail fast, rather than discovering a typo only when that code path runs in production):

```
@Entity
@NamedQuery(
    name = "Student.findByMinGpa",
    query = "select s from Student s where s.gpa >= :minGpa order by s.gpa desc"
)
@NamedQuery(
    name = "Student.countActive",
    query = "select count(s) from Student s where s.active = true"
)
public class Student {
    // ... fields ...
}
```

```
List<Student> honorRoll = em.createNamedQuery("Student.findByMinGpa", Student.class)
        .setParameter("minGpa", 3.5)
        .getResultList();
```

```
Long activeCount = em.createNamedQuery("Student.countActive", Long.class)
        .getSingleResult();
```

`getSingleResult()` vs. `getResultList()`

```
// Expects EXACTLY one row — throws NoResultException if none, NonUniqueResultException if more
Student one = em.createQuery("select s from Student s where s.email = :email", Student.class)
        .setParameter("email", "unique@example.com")
        .getSingleResult();

// Safer for "may or may not exist" lookups — returns an empty list rather than throwing
List<Student> maybe = em.createQuery("select s from Student s where s.email = :email", Student.
        .setParameter("email", "maybe@example.com")
        .getResultList();
Optional<Student> found = maybe.stream().findFirst();
```

Pagination

```
List<Student> page = em.createQuery("select s from Student s order by s.name", Student.class)
        .setFirstResult(20)  // skip the first 20 rows (OFFSET)
        .setMaxResults(10)   // return at most 10 rows (LIMIT)
        .getResultList();
```

Hibernate translates `setFirstResult`/`setMaxResults` into the correct dialect-specific pagination SQL (`LIMIT ... OFFSET ...` on H2/MySQL/PostgreSQL, `ROWNUM`/`FETCH FIRST` on Oracle, etc.) — another example of the database-portability benefit from Lesson 1.3.1.

8.2 Demonstration for JPQL

A complete, runnable demonstration combining everything above, using the same entities built up across Lessons 1–7:

```
package com.example.hibernatecourse;

import com.example.hibernatecourse.entity.Student;
import com.example.hibernatecourse.dto.StudentSummaryDTO;
import jakarta.persistence.*;

import java.util.List;

public class JpqlDemo {
    public static void main(String[] args) {
        EntityManagerFactory emf = Persistence.createEntityManagerFactory("hibernateCoursePU");
        EntityManager em = emf.createEntityManager();

        try {
            seedData(em);

            // 1. Restriction + named parameter
            List<Student> honorRoll = em.createQuery(
```

```
                "select s from Student s where s.gpa >= :minGpa order by s.gpa desc", Stude
                .setParameter("minGpa", 3.5)
                .getResultList();
        System.out.println("Honor roll: " + honorRoll.size());

        // 2. JOIN FETCH — avoid N+1 when iterating enrollments
        List<Student> withEnrollments = em.createQuery(
                "select distinct s from Student s left join fetch s.enrollments", Student.c
                .getResultList();
        withEnrollments.forEach(s ->
                System.out.println(s.getName() + " -> " + s.getEnrollments().size() + " enr

        // 3. Aggregate query
        Long totalActive = em.createQuery(
                "select count(s) from Student s where s.active = true", Long.class)
                .getSingleResult();
        System.out.println("Active students: " + totalActive);

        // 4. DTO projection
        List<StudentSummaryDTO> summaries = em.createQuery(
                "select new com.example.hibernatecourse.dto.StudentSummaryDTO(s.name, s.gpa
                StudentSummaryDTO.class)
                .getResultList();
        summaries.forEach(dto -> System.out.println(dto.getName() + ": " + dto.getGpa()));

        // 5. Pagination
        List<Student> page1 = em.createQuery(
                "select s from Student s order by s.name", Student.class)
                .setFirstResult(0)
                .setMaxResults(5)
                .getResultList();
        System.out.println("Page 1 size: " + page1.size());

        // 6. Bulk update
        em.getTransaction().begin();
        int deactivated = em.createQuery(
                "update Student s set s.active = false where s.gpa < :threshold")
                .setParameter("threshold", 2.0)
                .executeUpdate();
        em.getTransaction().commit();
        System.out.println("Deactivated: " + deactivated);

    } finally {
        em.close();
        emf.close();
    }
}

private static void seedData(EntityManager em) {
    em.getTransaction().begin();
    em.persist(new Student("Aditi Sharma", "aditi@example.com"));
    em.persist(new Student("Rohan Das", "rohan@example.com"));
    em.persist(new Student("Meera Nair", "meera@example.com"));
```

```
        em.getTransaction().commit();
    }
}
```

Expected behavior when run (matching the Hibernate SQL logging enabled since Lesson 1):

1. Three `INSERT` statements from `seedData`.
2. A single `SELECT ... WHERE gpa >= ? ORDER BY gpa DESC` for the honor roll.
3. A single `SELECT ... LEFT JOIN ...` for the fetch-joined students (no N+1, per Lesson 3.4/5.2.1).
4. A single `SELECT count(*) ...` for the aggregate.
5. A `SELECT` mapped directly into `StudentSummaryDTO` objects via the constructor expression (Lesson 5.4).
6. A paginated `SELECT ... LIMIT 5 OFFSET 0` (or dialect equivalent).
7. A single bulk `UPDATE ... WHERE gpa < ?` statement, bypassing the persistence context (Lesson 5.2.2).

Key Takeaways from Lesson 8

- **JPQL is the standardized counterpart to HQL** — nearly all HQL syntax you learned in Lesson 5 (restrictions, named parameters, joins, `JOIN FETCH`, aggregates, constructor/DTO projections, bulk updates) works unchanged as JPQL, executed via `EntityManager.createQuery(...)` instead of `Session.createQuery(...)`.
- **`@NamedQuery`** predefines and validates JPQL at startup, catching syntax errors early and centralizing query definitions on the entity itself.
- **`getSingleResult()`** expects exactly one row (throws otherwise); **`getResultList()`** is the safer choice when a match may or may not exist.
- **Pagination** (`setFirstResult`/`setMaxResults`) is translated into the correct dialect-specific SQL automatically, preserving the database-portability promise from Lesson 1.
- Because Hibernate is a full JPA provider, everything you've learned across all eight lessons — entity mapping, the persistence lifecycle, associations, transactions, and querying — is accessible through *either* the Hibernate-native API *or* the standardized JPA API, and real-world Spring/Spring Boot applications typically favor the JPA (`EntityManager`/JPQL/Spring Data JPA) surface for portability, while still being free to "unwrap" to native Hibernate features (Filters, Interceptors, `@BatchSize`) when needed.

Course Complete

This concludes the full 8-lesson deep dive:

1. Introduction to ORM and Its Need
2. The Persistence Life Cycle
3. Hibernate Association and Collection Mapping
4. Transaction Management
5. Searches and Queries
6. Caching
7. Java Persistence API (JPA)
8. JPQL

Across these lessons, one running example (`Student`, `Course`, `Enrollment`, `Address`, `Payment`) was built up incrementally into a fully working Hibernate/JPA application, covering ORM theory, the object lifecycle, every association/inheritance mapping strategy, transaction management (programmatic and declarative), HQL/JPQL/Criteria querying, N+1 mitigation, two-level caching, and the JPA specification layer — mirroring the structure and terminology used throughout the official Hibernate ORM documentation at hibernate.org.

Addendum: Optimistic and Pessimistic Locking in Hibernate

> This addendum extends Lesson 3 (Cascade Types) and Lesson 7 (JPA `@Version`) with a full hands-on deep dive into **concurrency control** — how Hibernate prevents two transactions from silently overwriting each other's changes to the same row.

Why Locking Matters: The Lost Update Problem

Consider two users editing the same `Student` record at the same time, without any locking:

```
Time  Transaction A                          Transaction B
----  -------------------------------------  -------------------------------------
t1    SELECT gpa FROM STUDENT WHERE id=1
      -> reads gpa = 3.0
t2                                           SELECT gpa FROM STUDENT WHERE id=1
                                              -> reads gpa = 3.0
t3    UPDATE STUDENT SET gpa = 3.5 WHERE id=1
      COMMIT                                  (A's change is now in the DB)
t4                                           UPDATE STUDENT SET gpa = 3.2 WHERE id=1
                                              COMMIT
```

Result: B's `UPDATE` silently overwrites A's change — the database ends up at `gpa = 3.2`, and A's update to `3.5` is **lost** entirely, with no error, no warning, nothing. This is the classic **lost update problem**, and it's exactly what locking strategies exist to prevent.

Hibernate offers two fundamentally different strategies to solve this:

Strategy	Core Idea	Cost	Best For
Optimistic Locking	Assume conflicts are rare; detect them at commit time via a version check.	Cheap — no DB locks held.	Web apps, high-concurrency reads, "think time" between read and write (e.g., editing a form).
Pessimistic Locking	Assume conflicts are likely; physically lock the row at read time so no one else can touch it.	Expensive — holds DB-level locks, reduces concurrency, risk of deadlocks.	Short, high-contention critical sections (e.g., decrementing inventory stock, financial ledger updates).

Part 1: Optimistic Locking

The Mechanism: `@Version`

Optimistic locking doesn't take any database lock at all. Instead, every entity carries a **version** column, and every `UPDATE` Hibernate generates includes `WHERE version = <the version it read>` — if that row's version has since changed (meaning someone else updated it), **zero rows match**, and Hibernate throws an exception instead of silently overwriting.

```java
package com.example.hibernatecourse.entity;

import jakarta.persistence.*;

@Entity
@Table(name = "STUDENT")
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    private double gpa;

    @Version
    private int version; // Hibernate manages this field entirely — never set it yourself

    public Student() {}
    public Student(String name, double gpa) {
        this.name = name;
        this.gpa = gpa;
    }
    // getters/setters (no setter needed for 'version' in application code)
}
```

The generated `UPDATE` SQL for this entity now always looks like:

```sql
update STUDENT
set name=?, gpa=?, version=?
where id=? and version=?     -- the OLD version value, from when the entity was loaded
```

If `version` in the `WHERE` clause no longer matches the row in the database (because another transaction already updated it, bumping the version), the `UPDATE` affects **0 rows**, and Hibernate raises:

```
jakarta.persistence.OptimisticLockException
  (caused by: org.hibernate.StaleObjectStateException / StaleStateException)
```

Hands-On: Reproducing and Handling an Optimistic Lock Conflict

```java
package com.example.hibernatecourse;

import com.example.hibernatecourse.entity.Student;
import com.example.hibernatecourse.util.HibernateUtil;
import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.StaleObjectStateException;
import org.hibernate.Transaction;

public class OptimisticLockDemo {
    public static void main(String[] args) {
        SessionFactory sf = HibernateUtil.getSessionFactory();
```

```
        Long studentId = seedStudent(sf);

        // --- Simulate two users loading the SAME row in separate sessions ---
        Session sessionA = sf.openSession();
        Student studentA = sessionA.get(Student.class, studentId); // version = 0

        Session sessionB = sf.openSession();
        Student studentB = sessionB.get(Student.class, studentId); // ALSO version = 0

        // --- User A commits their change first ---
        Transaction txA = sessionA.beginTransaction();
        studentA.setGpa(3.5);
        txA.commit(); // succeeds — version bumps from 0 -> 1 in the DB
        sessionA.close();
        System.out.println("User A committed successfully. GPA = 3.5, version now 1.");

        // --- User B, unaware of A's change, tries to commit their own edit ---
        Transaction txB = sessionB.beginTransaction();
        try {
            studentB.setGpa(3.2); // studentB still thinks version = 0
            txB.commit();        // Hibernate issues UPDATE ... WHERE id=? AND version=0 -> 0 R
            System.out.println("User B committed (this line should NOT print)");
        } catch (org.hibernate.StaleStateException e) {
            System.out.println("Conflict detected! User B's change was based on stale data.");
            txB.rollback();
            // Real application response: reload the current state, show the conflict to the us
            // and let them decide whether to re-apply their change on top of the fresh data.
        } finally {
            sessionB.close();
        }

        HibernateUtil.shutdown();
    }

    private static Long seedStudent(SessionFactory sf) {
        try (Session session = sf.openSession()) {
            Transaction tx = session.beginTransaction();
            Student s = new Student("Neha Kulkarni", 3.0);
            session.persist(s);
            tx.commit();
            return s.getId();
        }
    }
}
```

Console output:

```
Hibernate: update STUDENT set gpa=?, name=?, version=? where id=? and version=?
User A committed successfully. GPA = 3.5, version now 1.
Hibernate: update STUDENT set gpa=?, name=?, version=? where id=? and version=?
Conflict detected! User B's change was based on stale data.
```

Note that Hibernate **still attempts** User B's `UPDATE` (you'll see the SQL fire) — the conflict is only discovered because the affected-row-count comes back as `0`, at which point Hibernate raises the exception on your behalf. This is the essence of optimistic locking: cheap, no locks

held while "thinking," but the conflict is only caught at the very last moment (commit/flush time).

Handling the Conflict Gracefully (typical web-app pattern)

```
@Transactional
public void updateGpa(Long studentId, double newGpa, int expectedVersion) {
    Student s = em.find(Student.class, studentId);
    if (s.getVersion() != expectedVersion) {
        throw new ConcurrentModificationException(
            "This record was changed by someone else. Please refresh and try again.");
    }
    s.setGpa(newGpa);
    // em.merge / dirty checking handles the rest; @Version still protects against a race
    // between this check and the actual commit.
}
```

In a real web application, the `expectedVersion` typically comes back from a hidden form field rendered when the edit page was first loaded — so the check happens both at the application level (friendlier error message) **and** at the database level via `@Version` (the actual safety net, since the application-level check alone has its own race window).

Explicit Optimistic Lock Modes (forcing a version check even on read-only paths)

```
// Force Hibernate to verify the version hasn't changed, even for an entity you're not updating
// e.g., to make sure a "confirm" step is acting on data that's still current.
Student s = session.get(Student.class, studentId, LockModeType.OPTIMISTIC);

// OPTIMISTIC_FORCE_INCREMENT: bump the version even if no fields actually changed —
// useful for signaling "this entity's associated data changed" without altering the entity's o
Student s2 = session.get(Student.class, studentId, LockModeType.OPTIMISTIC_FORCE_INCREMENT);
```

Part 2: Pessimistic Locking

The Mechanism: Database-Level Row Locks

Pessimistic locking takes an actual lock **in the database** at the moment you read the row, using SQL's `SELECT ... FOR UPDATE` (or dialect equivalent) — any other transaction trying to read (with a lock) or write that same row **blocks** until the lock is released (commit/rollback).

```
Session session = sf.openSession();
Transaction tx = session.beginTransaction();

// Acquires a DB-level exclusive lock on this row immediately
Student s = session.get(Student.class, studentId, LockModeType.PESSIMISTIC_WRITE);

s.setGpa(s.getGpa() + 0.1); // safe: no one else can modify this row until we commit

tx.commit(); // releases the lock
session.close();
```

Generated SQL (H2/PostgreSQL-style):

```
select s1_0.id, s1_0.gpa, s1_0.name, s1_0.version
from STUDENT s1_0
where s1_0.id = ?
for update
```

`LockModeType` Options for Pessimistic Locking

Mode	Behavior
`PESSIMISTIC_READ`	Shared lock — allows other transactions to also read-lock the row, but blocks any write.
`PESSIMISTIC_WRITE`	Exclusive lock — blocks all other transactions (read-lock or write) on this row until commit/rollback. Most common.
`PESSIMISTIC_FORCE_INCREMENT`	Exclusive lock, and also forces a `@Version` increment (combines both strategies).

Hands-On: Reproducing Pessimistic Lock Blocking

```
package com.example.hibernatecourse;

import com.example.hibernatecourse.entity.Student;
import com.example.hibernatecourse.util.HibernateUtil;
import jakarta.persistence.LockModeType;
import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Transaction;

import java.util.concurrent.CountDownLatch;

public class PessimisticLockDemo {
    public static void main(String[] args) throws InterruptedException {
        SessionFactory sf = HibernateUtil.getSessionFactory();
        Long studentId = seedStudent(sf);

        CountDownLatch transactionAHasLock = new CountDownLatch(1);
        CountDownLatch letTransactionACommit = new CountDownLatch(1);

        // --- Thread A: acquires PESSIMISTIC_WRITE and holds it for a few seconds ---
        Thread threadA = new Thread(() -> {
            try (Session session = sf.openSession()) {
                Transaction tx = session.beginTransaction();
                System.out.println("[A] Acquiring PESSIMISTIC_WRITE lock...");
                Student s = session.get(Student.class, studentId, LockModeType.PESSIMISTIC_WRIT
                System.out.println("[A] Lock acquired. Holding it...");
                transactionAHasLock.countDown();

                letTransactionACommit.await(); // wait until told to proceed

                s.setGpa(4.0);
                tx.commit();
                System.out.println("[A] Committed and released the lock.");
```

```
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        });

        // --- Thread B: tries to acquire the SAME lock and will BLOCK until A commits ---
        Thread threadB = new Thread(() -> {
            try {
                transactionAHasLock.await(); // wait until A definitely holds the lock first
                try (Session session = sf.openSession()) {
                    Transaction tx = session.beginTransaction();
                    System.out.println("[B] Requesting PESSIMISTIC_WRITE lock (should block)...
                    long start = System.currentTimeMillis();

                    Student s = session.get(Student.class, studentId, LockModeType.PESSIMISTIC_

                    long waited = System.currentTimeMillis() - start;
                    System.out.println("[B] Lock acquired after blocking for " + waited + " ms.
                    tx.commit();
                }
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        });

        threadA.start();
        threadB.start();

        Thread.sleep(3000); // let B sit blocked for 3 seconds, proving the lock is real
        System.out.println("[main] Releasing A's lock now...");
        letTransactionACommit.countDown();

        threadA.join();
        threadB.join();
        HibernateUtil.shutdown();
    }

    private static Long seedStudent(SessionFactory sf) {
        try (Session session = sf.openSession()) {
            Transaction tx = session.beginTransaction();
            Student s = new Student("Farhan Ali", 3.0);
            session.persist(s);
            tx.commit();
            return s.getId();
        }
    }
}
```

Expected console output:

```
[A] Acquiring PESSIMISTIC_WRITE lock...
[A] Lock acquired. Holding it...
[B] Requesting PESSIMISTIC_WRITE lock (should block)...
[main] Releasing A's lock now...
```

```
[A] Committed and released the lock.
[B] Lock acquired after blocking for ~3000 ms. gpa=4.0
```

Thread B's `session.get(..., PESSIMISTIC_WRITE)` call **physically blocks** for roughly 3 seconds — the exact duration Thread A held the lock — proving the row was genuinely locked at the database level, not just checked after the fact (as optimistic locking does).

Avoiding Indefinite Blocking: Lock Timeouts

Holding a thread blocked forever waiting for a lock is dangerous (risk of application-wide thread starvation, or deadlocks). Always set a timeout:

```
Map<String, Object> properties = new HashMap<>();
properties.put("jakarta.persistence.lock.timeout", 5000); // milliseconds

try {
    Student s = session.get(Student.class, studentId, LockModeType.PESSIMISTIC_WRITE, propertie
} catch (jakarta.persistence.LockTimeoutException e) {
    System.out.println("Could not acquire lock within timeout — try again later.");
}
```

Or via JPQL/HQL query hints:

```
Student s = em.createQuery("select s from Student s where s.id = :id", Student.class)
        .setParameter("id", studentId)
        .setLockMode(LockModeType.PESSIMISTIC_WRITE)
        .setHint("jakarta.persistence.lock.timeout", 5000)
        .getSingleResult();
```

Deadlock Awareness

Pessimistic locking introduces real **deadlock risk**: if Transaction A locks row 1 then tries to lock row 2, while Transaction B locks row 2 then tries to lock row 1, both block forever waiting on each other. Most databases detect this automatically and abort one transaction with a deadlock exception (`org.hibernate.exception.LockAcquisitionException` in Hibernate) — your application code should be prepared to **catch this and retry** the whole transaction:

```
int maxRetries = 3;
for (int attempt = 1; attempt <= maxRetries; attempt++) {
    try {
        doWorkRequiringLock();
        break; // success
    } catch (org.hibernate.exception.LockAcquisitionException e) {
        if (attempt == maxRetries) throw e;
        System.out.println("Deadlock detected, retrying (attempt " + attempt + ")...");
    }
}
```

Mitigation practice: always acquire locks on multiple rows in a **consistent, agreed-upon order** (e.g., always lowest-ID-first) across your entire codebase — this alone eliminates the classic circular-wait deadlock pattern shown above.

Optimistic vs. Pessimistic: Side-by-Side Comparison

	Optimistic (`@Version`)	Pessimistic (`PESSIMISTIC_WRITE/READ`)
When conflict is detected	At commit/flush time	Immediately at read time (blocks)
Database locks held	None	Yes, for the duration of the transaction
Concurrency (throughput)	High — no one is blocked while "thinking"	Lower — competing transactions serialize
Risk	Lost work discovered late (user must retry)	Deadlocks, thread starvation if timeouts aren't set
Typical use case	Web forms, long user "think time" between load and save	Short critical sections: inventory decrement, seat booking, ledger balance updates
Requires schema change	Yes — a `version` (or `@Column` timestamp-based) column	No — uses the database's native row-locking, no extra column

> **Rule of thumb from real-world practice:** default to **optimistic locking** for anything resembling a web form or a slow human-in-the-loop update, since it scales far better under load. Reach for **pessimistic locking** only for tight, fast, code-only critical sections where correctness under high contention (e.g., two requests trying to book the last seat at the same instant) matters more than raw throughput — and always pair it with an explicit timeout and a retry strategy.

Key Takeaways from This Addendum

- **Optimistic locking** (`@Version`) detects conflicts cheaply, at commit time, by including the version in the `UPDATE`'s `WHERE` clause — zero rows affected means someone else already changed the data, raising `OptimisticLockException/StaleStateException`.
- **Pessimistic locking** (`LockModeType.PESSIMISTIC_READ/PESSIMISTIC_WRITE`) takes a real database row lock immediately via `SELECT ... FOR UPDATE`, physically blocking other transactions until commit/rollback.
- Both strategies solve the same underlying **lost update problem**, but trade off differently between throughput (optimistic wins) and immediate correctness under contention (pessimistic wins).
- Always configure a **lock timeout** for pessimistic locks, and be prepared to **catch and retry** on `LockAcquisitionException` (deadlocks) — never let a thread wait indefinitely for a lock.
- Default to optimistic locking unless you have a specific, narrow, high-contention critical section that justifies the cost of pessimistic locks.

Addendum: The N+1 Query Problem — Deep Dive on Resolution Strategies

> This addendum extends Lesson 3 (§3.4) and Lesson 5 (§5.2.1) with a consolidated, hands-on reference covering every practical way to detect and resolve the N+1 query problem.

What Is the N+1 Query Problem?

Loading **1** query's worth of parent entities, and then separately triggering **N** additional queries — one per parent — to fetch a lazily-loaded association, instead of retrieving everything in one efficient round trip. The name comes from the query count: **1** (for the parents) **+ N** (one per parent for the association) = **N+1** total queries where 1 or 2 would have sufficed.

Reproducing It

```
List<Student> students = session.createQuery("from Student", Student.class).list(); // query #1

for (Student s : students) {
    System.out.println(s.getName() + " has " + s.getEnrollments().size() + " enrollments");
    // ^ triggers ONE additional SELECT per student, because enrollments is FetchType.LAZY
}
```

Console output (the smoking gun):

```
select s1_0.id, s1_0.name from STUDENT s1_0                                    -- the "1"
select e1_0.student_id, e1_0.id, e1_0.semester
    from ENROLLMENT e1_0 where e1_0.student_id = ?                             -- repeated N ti
select e1_0.student_id, e1_0.id, e1_0.semester
    from ENROLLMENT e1_0 where e1_0.student_id = ?
... (N times total)
```

With 100 students, that's **101 round trips** to the database instead of 1–2 — and this scales linearly, so 1,000 students means 1,001 queries. This is consistently the single most common real-world cause of slow Hibernate/JPA applications, precisely because it's invisible in the code (`s.getEnrollments().size()` looks harmless) and only shows up as a performance problem under realistic data volumes, often missed in small dev/test datasets.

Why It Happens

By default, `@OneToMany`/`@ManyToMany` associations are `FetchType.LAZY` (Lesson 2.6) — a sensible default, since eagerly loading every association on every entity load would be wasteful in the opposite direction. The problem isn't laziness itself; it's **accessing a lazy association inside a loop over many parent entities** without telling Hibernate in advance that you'll need it.

Detecting the N+1 Problem

Before fixing it, confirm it's actually happening:

1. **Enable SQL logging** (`hibernate.show_sql=true`, or better, a proper logger for `org.hibernate.SQL` at `DEBUG`) and watch for repeated, near-identical `SELECT` statements differing only in a bound parameter value.
2. **Enable Hibernate statistics** and inspect query counts:

```
sessionFactory.getStatistics().setStatisticsEnabled(true);
// ... run the suspect code path ...
System.out.println("Query count: " + sessionFactory.getStatistics().getQueryExecutionCount(
```

3. **Use a dedicated tool** in real projects — libraries like Hypersistence's `db-util` or IDE/APM query-count assertions in integration tests (`assertQueryCount(2)`) catch regressions automatically in CI, before they reach production.

Resolution Strategies

1. `JOIN FETCH` in HQL/JPQL (the most common, per-query fix)

Explicitly eager-load the association for this **specific query only**, via a SQL `JOIN`:

```
List<Student> students = session.createQuery(
        "select distinct s from Student s left join fetch s.enrollments", Student.class)
        .list(); // now just 1 query total
```

- `distinct` prevents duplicate `Student` references at the object level (a one-to-many `JOIN` returns one row per child row).
- Best used when a **specific code path** knows in advance it will need the association — doesn't affect other queries against the same entity that don't need it.
- **Caution:** fetch-joining more than one **collection** association (`List`-typed especially) in a single query can trigger a `MultipleBagFetchException` or a Cartesian-product row explosion; prefer `Set` (Lesson 3.1) and fetch-join collections one at a time, or combine with a `*-to-one` fetch join safely:

```
session.createQuery(
    "select distinct s from Student s " +
    "left join fetch s.enrollments e " +
    "left join fetch e.course", // safe: 'course' is *-to-one, doesn't multiply the collection
    Student.class).list();
```

2. JPA Entity Graphs (declarative, reusable, standardized)

An **Entity Graph** lets you declare, separately from the query itself, which associations to eagerly fetch — reusable across multiple query methods and standard across JPA providers (not Hibernate-specific):

```
EntityGraph<Student> graph = em.createEntityGraph(Student.class);
graph.addAttributeNodes("enrollments");

List<Student> students = em.createQuery("select s from Student s", Student.class)
        .setHint("jakarta.persistence.fetchgraph", graph)
        .getResultList(); // 1 query, enrollments eagerly joined
```

Named entity graphs (declared once on the entity, referenced by name anywhere):

```
@Entity
@NamedEntityGraph(
```

```
    name = "Student.withEnrollments",
    attributeNodes = @NamedAttributeNode("enrollments")
)
public class Student { ... }
```

```
EntityGraph<?> graph = em.getEntityGraph("Student.withEnrollments");
em.createQuery("select s from Student s", Student.class)
  .setHint("jakarta.persistence.fetchgraph", graph)
  .getResultList();
```

Entity graphs shine when the **same** fetch strategy needs to apply across several different query methods (e.g., a repository with multiple `findBy...` methods) without duplicating `JOIN FETCH` HQL in each one.

3. `@BatchSize` (turns N queries into a small constant number)

Rather than eagerly joining, this **batches** the follow-up lazy-load queries using `WHERE ... IN (?, ?, ...)`:

```
@Entity
public class Student {
    @OneToMany(mappedBy = "student")
    @BatchSize(size = 25)
    private Set<Enrollment> enrollments;
}
```

With 100 students and `@BatchSize(25)`, Hibernate issues `⌈100 / 25⌉ = 4` follow-up queries instead of 100:

```
select ... from ENROLLMENT where student_id in (?, ?, ..., up to 25 ids)
-- repeated 4 times total instead of 100 times
```

Global default (fixes N+1 application-wide, without annotating every entity):

```
hibernate.default_batch_fetch_size=25
```

This is often the **single highest-leverage fix** in an existing, large codebase — one configuration line mitigates N+1 across the entire domain model retroactively, without touching every entity or every query.

4. `FetchMode.SUBSELECT`

Loads the collection for **every currently-loaded parent** in one extra query, using a subselect matching all their identifiers:

```
@OneToMany(mappedBy = "student")
@Fetch(FetchMode.SUBSELECT)
private Set<Enrollment> enrollments;
```

```
select ... from ENROLLMENT where student_id in (select id from STUDENT where <original query's
```

Total queries: **2** (one for the parents, one subselect for all their children) regardless of how many parents there are — even better than `@BatchSize` for very large result sets, though less flexible since it reruns (a form of) the original query's filter condition rather than a simple ID list.

5. DTO / Constructor-Expression Projections

Sidestep the problem entirely by never loading the association as a managed entity collection — query exactly the flattened data you need in one pass (Lesson 5.4):

```
List<StudentEnrollmentDTO> rows = session.createQuery(
        "select new com.example.hibernatecourse.dto.StudentEnrollmentDTO(s.name, e.semester) "
        "from Student s join s.enrollments e",
        StudentEnrollmentDTO.class)
        .list(); // 1 query, no lazy association ever touched
```

Ideal for read-only reporting/listing use cases where you don't need full entity objects (with their own lifecycle, dirty-checking overhead, etc.) at all.

6. Criteria API Fetch Joins

The programmatic equivalent of strategy #1, for dynamically-built queries (Lesson 5.3):

```
CriteriaBuilder cb = session.getCriteriaBuilder();
CriteriaQuery<Student> cq = cb.createQuery(Student.class);
Root<Student> root = cq.from(Student.class);
root.fetch("enrollments", JoinType.LEFT); // fetch join, not just a filtering join

cq.select(root).distinct(true);
List<Student> students = session.createQuery(cq).list();
```

Choosing the Right Strategy

Strategy	Total Queries	Scope	Best When
`JOIN FETCH` (HQL/JPQL)	1	Single query	You know exactly what one specific query needs, and it's a single collection.
Entity Graph	1	Reusable across queries	JPA-standard codebases; the same fetch shape is needed by multiple query methods.
`@BatchSize`	⌈N / batchSize⌉	Per-entity, or global default	Access patterns are unpredictable, or you need a blanket fix across a large, existing codebase.
`FetchMode.SUBSELECT`	2	Per-entity	Loading many parents where (almost) all of them will need the same collection.
DTO projection	1	Per query	Read-only/reporting use cases that don't need full entity objects at all.

Strategy	Total Queries	Scope	Best When
Criteria fetch join	1	Single query, programmatic	Dynamically-built queries (optional filters) that also need eager association loading.

What *not* to do: switch the association's default to `FetchType.EAGER` globally as a "fix." This doesn't eliminate the extra queries (or turns them into a forced `JOIN`/extra `SELECT` on *every* load) — it just makes the cost unconditional on **every single query path** that touches that entity, including ones that never needed the association in the first place, often making overall performance *worse*, not better. Always fix N+1 at the **query level** (per strategy above), not by changing the entity's default fetch type.

Key Takeaways

- N+1 arises from accessing a **lazy association inside a loop** over many parent entities — 1 query for the parents plus N follow-up queries for the association.
- Detect it via SQL logging or `SessionFactory.getStatistics().getQueryExecutionCount()` before assuming a fix is needed.
- Six practical resolutions exist: **`JOIN FETCH`**, **Entity Graphs**, **`@BatchSize`** (including a global default), **`FetchMode.SUBSELECT`**, **DTO projections**, and **Criteria API fetch joins** — each with different trade-offs in query count, reusability, and flexibility.
- Never "fix" N+1 by flipping an association to `FetchType.EAGER` globally — that only relocates the cost to every query path, rather than eliminating it.

Addendum: Spring Data JPA Repositories and Native SQL Queries

This addendum fills the two gaps between the Service layer (Lesson 4.2) and the `EntityManager` (Lesson 7.6): the **Spring Data JPA repository abstraction**, and **native SQL** as an alternative to JPQL (Lesson 8) for complex or performance-critical queries.

Part 1: Spring Data JPA Repositories

Where This Fits in the Request Flow

`Controller → Service (@Transactional) → Repository (Spring Data JPA) → EntityManager → Hibernate → J`

Everything in Lesson 4's `StudentService` talked to `Session`/`EntityManager` directly. In real Spring applications, the **DAO/data-access layer is almost never hand-written** — Spring Data JPA generates it for you at runtime from a simple **interface**, eliminating boilerplate CRUD code entirely.

The Core Idea: You Write an Interface, Spring Writes the Implementation

```java
package com.example.hibernatecourse.repository;

import com.example.hibernatecourse.entity.Student;
import org.springframework.data.jpa.repository.JpaRepository;

public interface StudentRepository extends JpaRepository<Student, Long> {
    // No implementation needed — Spring generates a dynamic proxy at startup
}
```

That single interface, extending `JpaRepository<Student, Long>` (entity type, ID type), already provides:

Inherited Method	Equivalent `EntityManager` Call (Lesson 7.6)
`save(Student s)`	`em.persist(s)` or `em.merge(s)` (Spring picks based on whether the ID is set)
`findById(Long id)`	`em.find(Student.class, id)` — returns `Optional<Student>`
`findAll()`	`em.createQuery("from Student", Student.class).getResultList()`
`deleteById(Long id)`	`em.remove(em.find(Student.class, id))`
`count()`	`em.createQuery("select count(s) from Student s", Long.class).getSingleResult()`
`existsById(Long id)`	Existence check without loading the full entity
`findAll(Pageable p)`	Paginated `getResultList()` (Lesson 8.1's `setFirstResult`/`setMaxResults`)

Using it in a Service (tying back to Lesson 4.2's declarative transactions):

```java
@Service
public class StudentService {

    private final StudentRepository studentRepository;

    public StudentService(StudentRepository studentRepository) { // constructor injection
        this.studentRepository = studentRepository;
    }

    @Transactional
    public Student registerStudent(String name, String email) {
        Student s = new Student(name, email);
        return studentRepository.save(s); // no EntityManager code anywhere in sight
    }

    @Transactional(readOnly = true)
    public Optional<Student> findStudent(Long id) {
        return studentRepository.findById(id);
    }
}
```

Derived Query Methods (query logic generated from the method name)

Spring Data JPA parses the **method name itself** and generates the matching JPQL automatically — no query string, no annotation, needed for common cases:

```java
public interface StudentRepository extends JpaRepository<Student, Long> {

    // Generates: select s from Student s where s.email = ?1
    Optional<Student> findByEmail(String email);

    // Generates: select s from Student s where s.gpa >= ?1 order by s.gpa desc
    List<Student> findByGpaGreaterThanEqualOrderByGpaDesc(double minGpa);

    // Generates: select s from Student s where s.name like ?1
    List<Student> findByNameStartingWith(String prefix);

    // Generates: select s from Student s where s.active = true and s.gpa > ?1
    List<Student> findByActiveTrueAndGpaGreaterThan(double gpa);

    // Generates: select count(s) from Student s where s.active = ?1
    long countByActive(boolean active);

    // Generates: delete from Student s where s.active = false
    void deleteByActiveFalse();
}
```

Common keyword building blocks: `findBy`, `countBy`, `existsBy`, `deleteBy` + property name + optional operator keywords (`GreaterThan`, `LessThanEqual`, `Between`, `Like`, `StartingWith`, `In`, `IsNull`, `True`, `False`) + optional `And`/`Or` combinators + optional `OrderBy...Asc/Desc`. This directly parallels the restrictions you wrote by hand in HQL/JPQL back in Lesson 5.1.2/8.1 — Spring is just generating that same JPQL from the method signature instead of you typing it.

`@Query` — Custom JPQL When Derived Names Get Unwieldy

Once a derived method name becomes long or awkward, drop down to explicit JPQL via `@Query` (this is exactly the JPQL from Lesson 8, just placed on a repository method instead of called via `EntityManager` directly):

```
public interface StudentRepository extends JpaRepository<Student, Long> {

    @Query("select s from Student s left join fetch s.enrollments where s.id = :id")
    Optional<Student> findWithEnrollments(@Param("id") Long id); // JOIN FETCH to avoid N+1 (Le

    @Query("select new com.example.hibernatecourse.dto.StudentSummaryDTO(s.name, s.gpa) from St
    List<StudentSummaryDTO> findAllSummaries(); // constructor-expression DTO projection (Lesso

    @Modifying
    @Query("update Student s set s.active = false where s.gpa < :threshold")
    int deactivateBelowGpa(@Param("threshold") double threshold); // bulk update (Lesson 5.2.2)
}
```

`@Modifying` is **required** on any `@Query` that isn't a `SELECT` (bulk `UPDATE`/`DELETE`) — without it, Spring Data JPA assumes every query returns a result set and will throw an exception at runtime.

`Pageable` and `Sort` (built-in pagination, wrapping Lesson 8.1's manual approach)

```
public interface StudentRepository extends JpaRepository<Student, Long> {
    Page<Student> findByActiveTrue(Pageable pageable);
}
```

```
Pageable pageable = PageRequest.of(0, 10, Sort.by("gpa").descending()); // page 0, size 10
Page<Student> page = studentRepository.findByActiveTrue(pageable);

System.out.println("Total elements: " + page.getTotalElements());
System.out.println("Total pages: " + page.getTotalPages());
List<Student> content = page.getContent();
```

Under the hood, this is exactly `setFirstResult`/`setMaxResults` (Lesson 8.1) plus a `count(*)` query for `getTotalElements()` — Spring Data JPA runs **two** queries (content + count) and wraps the results in a `Page` object for you.

What Spring Data JPA Is *Not*

It doesn't replace Hibernate/JPA — `JpaRepository` is a **thin abstraction layer over `EntityManager`**, which itself sits over Hibernate (Lesson 7.2). Every concept from Lessons 1–8 (entity lifecycle, dirty checking, cascade types, fetch strategies, N+1, caching) still applies exactly as described — Spring Data JPA only removes the boilerplate of writing the DAO interface's implementation by hand.

Part 2: Native SQL Queries

Why Drop Down to Native SQL?

JPQL/HQL (Lesson 8) is portable and covers the vast majority of query needs, but some things are either **impossible** or **impractical** in JPQL:

- Database-specific functions (e.g., PostgreSQL's `jsonb` operators, window functions with vendor-specific syntax, full-text search functions).
- Complex reporting queries (CTEs, recursive queries) that are awkward or unsupported in JPQL.
- Performance-critical queries where you've hand-tuned the exact SQL (query hints, specific index usage) and don't want the ORM's translation layer in the way.
- Working with existing hand-written SQL (e.g., migrating a legacy system incrementally).

The trade-off: **native SQL breaks database portability** (Lesson 1.3.1's dialect abstraction no longer applies) — you're back to SQL tied to one specific database vendor.

Native Queries via `EntityManager`

```
// Native SQL, mapped back onto the Student entity automatically (columns must match entity map
List<Student> students = em.createNativeQuery(
        "SELECT * FROM STUDENT WHERE gpa >= ?1 ORDER BY gpa DESC", Student.class)
        .setParameter(1, 3.5)
        .getResultList();
```

```
// Native SQL returning raw scalar/aggregate data (not mapped to an entity)
List<Object[]> rows = em.createNativeQuery(
        "SELECT s.name, COUNT(e.id) as enrollment_count " +
        "FROM STUDENT s LEFT JOIN ENROLLMENT e ON e.student_id = s.id " +
        "GROUP BY s.name")
        .getResultList();

for (Object[] row : rows) {
    System.out.println(row[0] + " -> " + row[1] + " enrollments");
}
```

`@SqlResultSetMapping` — Mapping Native SQL to DTOs Precisely

For complex native queries where you want type-safe DTO results (rather than untyped `Object[]`), define an explicit mapping:

```
@Entity
@SqlResultSetMapping(
    name = "StudentEnrollmentCountMapping",
    classes = @ConstructorResult(
        targetClass = StudentEnrollmentCountDTO.class,
        columns = {
            @ColumnResult(name = "student_name", type = String.class),
            @ColumnResult(name = "enrollment_count", type = Long.class)
        }
    )
)
public class Student { ... }
```

```
List<StudentEnrollmentCountDTO> results = em.createNativeQuery(
        "SELECT s.name AS student_name, COUNT(e.id) AS enrollment_count " +
        "FROM STUDENT s LEFT JOIN ENROLLMENT e ON e.student_id = s.id GROUP BY s.name",
        "StudentEnrollmentCountMapping")
        .getResultList();
```

Native Queries in Spring Data JPA Repositories

```
public interface StudentRepository extends JpaRepository<Student, Long> {

    @Query(value = "SELECT * FROM STUDENT WHERE gpa >= :minGpa ORDER BY gpa DESC", nativeQuery
    List<Student> findTopStudentsNative(@Param("minGpa") double minGpa);

    @Modifying
    @Query(value = "UPDATE STUDENT SET active = false WHERE gpa < :threshold", nativeQuery = tr
    int deactivateBelowGpaNative(@Param("threshold") double threshold);
}
```

Note the key difference from the JPQL version in Part 1: `nativeQuery = true`, and the query text now uses **actual table/column names** (`STUDENT`, `gpa`) rather than entity/property names (`Student`, `s.gpa`) — this is precisely the JPQL-vs-SQL distinction the infographic calls out.

Hibernate-Native Escape Hatch (beyond pure JPA)

If you need Hibernate-specific native query features (result transformers, named native query result set mappings defined via Hibernate's own annotations), you can always drop to the underlying `Session`:

```
Session session = em.unwrap(Session.class); // escape hatch from Lesson 7.6
List<Student> students = session.createNativeQuery(
        "SELECT * FROM STUDENT WHERE gpa >= :minGpa", Student.class)
        .setParameter("minGpa", 3.5)
        .list();
```

JPQL vs. Native SQL — Decision Guide

	JPQL/HQL (Lesson 8)	Native SQL
Portability	Database-agnostic (Hibernate translates via Dialect, Lesson 1.3.3)	Tied to one specific database's SQL dialect
Entity-awareness	Queries entity names/properties; results are managed entities by default	Requires explicit mapping (`Class` hint or `@SqlResultSetMapping`) to become entities
Feature coverage	Covers the vast majority of standard CRUD/reporting needs	Required for vendor-specific functions, CTEs, window functions, hand-tuned performance queries
Maintainability	Refactor-friendly (entity/property renames can be caught by tooling)	Column/table renames must be manually tracked across all native query strings
Recommendation	**Default choice** for essentially everything	**Escape hatch** for the specific cases JPQL can't handle well

---

Key Takeaways from This Addendum

- **Spring Data JPA repositories** (`JpaRepository<Entity, ID>`) sit between the Service layer and `EntityManager`, and Spring generates the DAO implementation at runtime — derived query methods parse method names into JPQL automatically, and `@Query` drops down to explicit JPQL (or native SQL with `nativeQuery = true`) when derived names aren't sufficient.
- `@Modifying` is mandatory on any non-`SELECT` `@Query` (bulk update/delete, Lesson 5.2.2's bulk operations exposed through the repository layer).
- `Pageable`/`Page<T>` wraps the manual `setFirstResult`/`setMaxResults` pagination from Lesson 8.1 into a higher-level, self-describing result object.
- **Native SQL** (`createNativeQuery`, `@Query(nativeQuery = true)`, `@SqlResultSetMapping`) is the deliberate escape hatch from JPQL/HQL's database portability — reach for it only when a specific query genuinely needs vendor-specific SQL features, and expect to lose portability across database vendors in exchange.
- Both features are **layers of convenience over the same underlying Hibernate/JPA machinery** covered in Lessons 1–8 and the earlier addenda — entity lifecycle, transactions, fetch strategies, caching, and locking all continue to apply unchanged underneath a `JpaRepository` or a native query.